Transformer

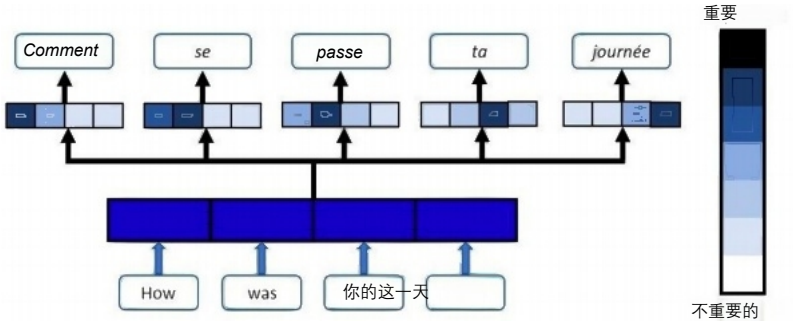
采用自注意力机制作为特征提取器

大纲；结构框架

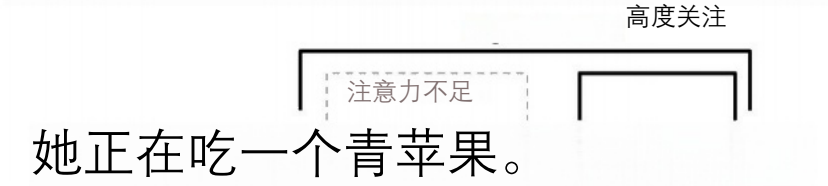
- 注意力与自注意力机制
- 变压器
- 使用BERT进行分类（示例代码）
- 自我关注（演示代码）

注意力与自我注意

·注意：在.....之间　输入　和　输出；结果　成分



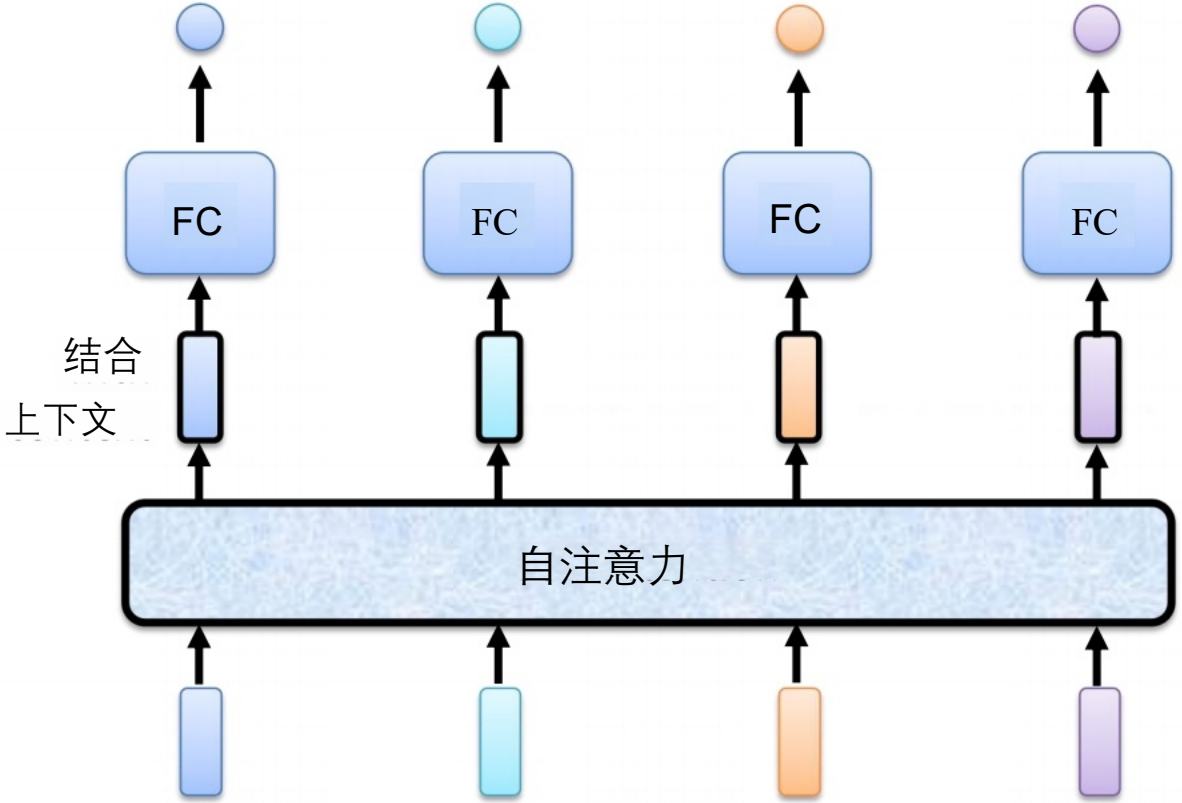
·自我关注：在输入元素内部

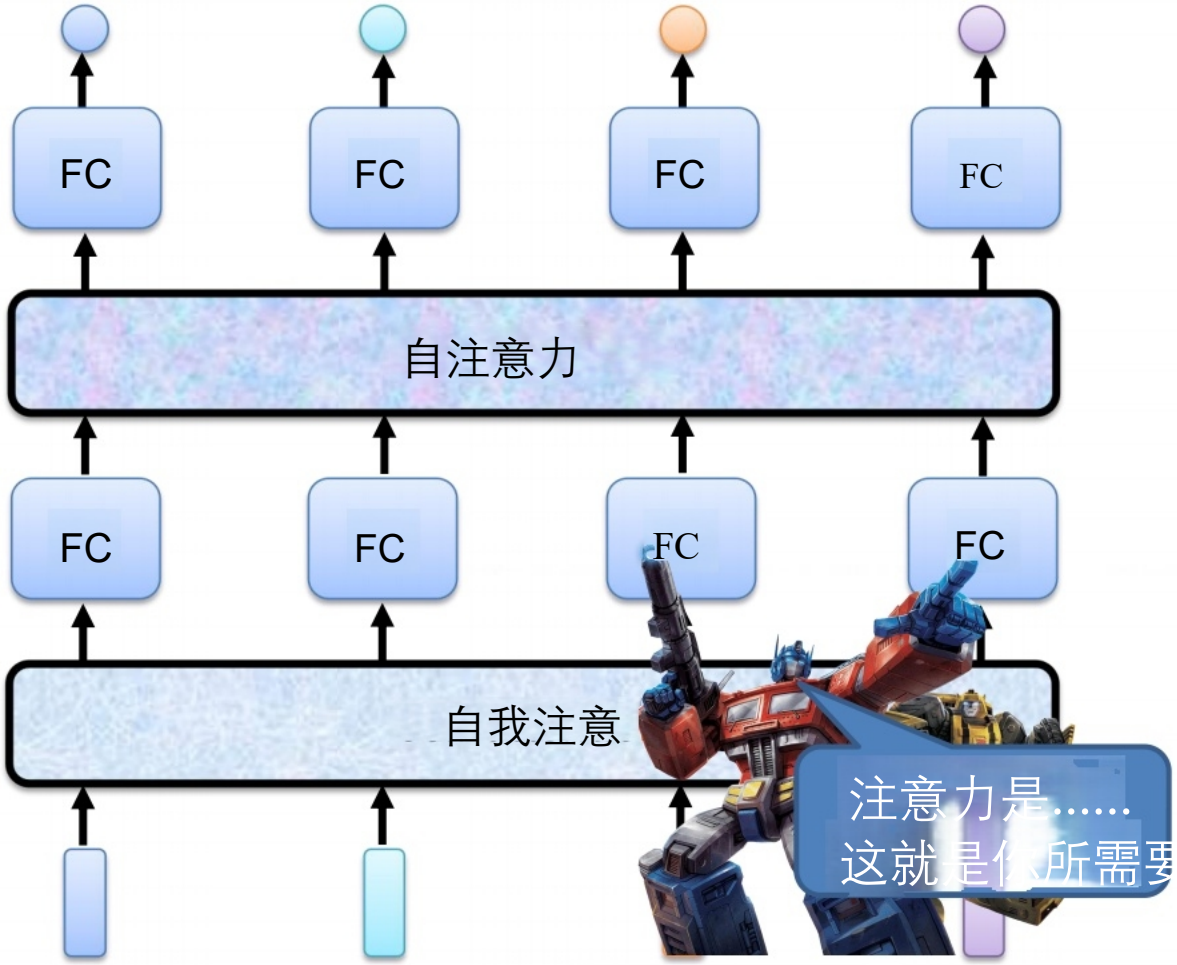


自己 注意；关心

- 自我关注是指每个词项应确保对句子中其他词项给予的关注程度的相对值。可将其视为一个表格，该表格以行和列的形式排列所有词项，其中第(i，j)个单元格表示第i行词项对第j列词项应提供的相对关注度。
- 自注意力机制是对注意力技术的一种创新性改进。在构建词嵌入时，它并非直接参考先前的隐藏向量，而是对所有其他词嵌入（包括句子中后续出现的词嵌入）进行加权组合。

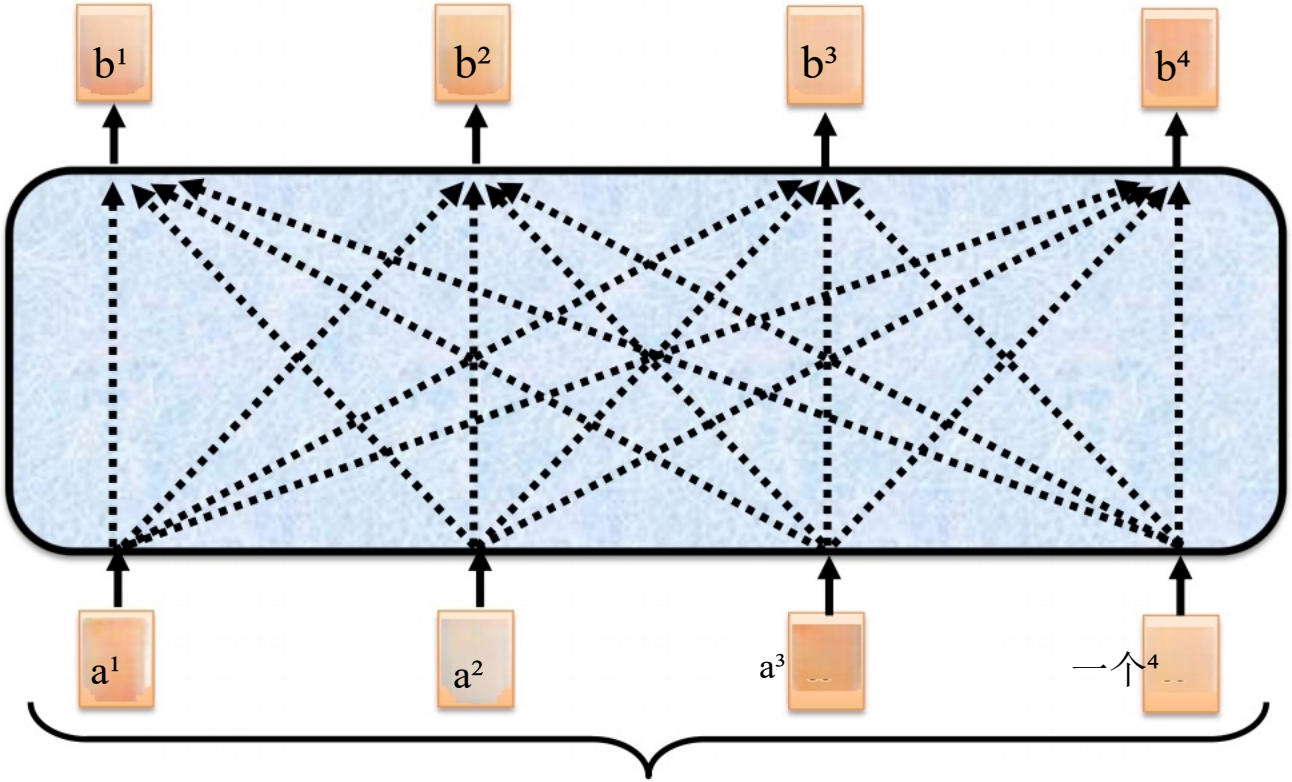
自注意力





https://arxiv.org/abs/1706.037

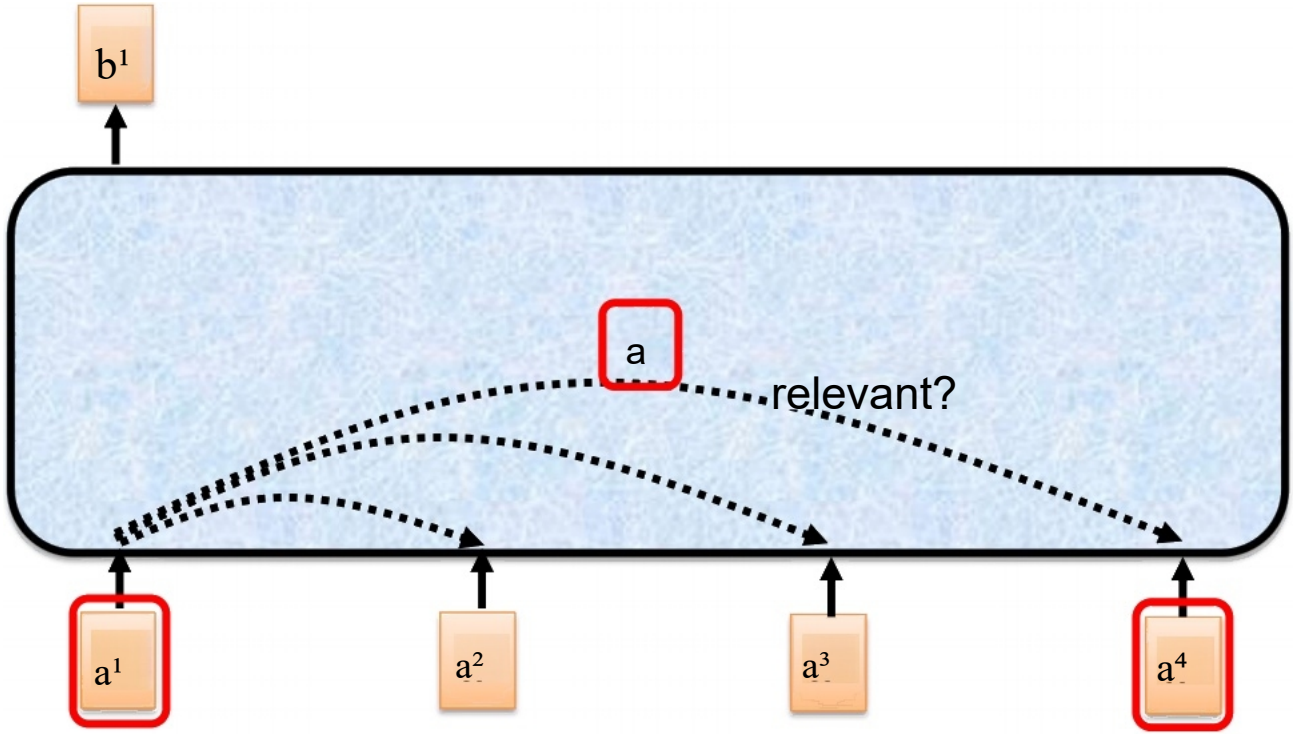
自注意力



可以是输入层或隐藏层

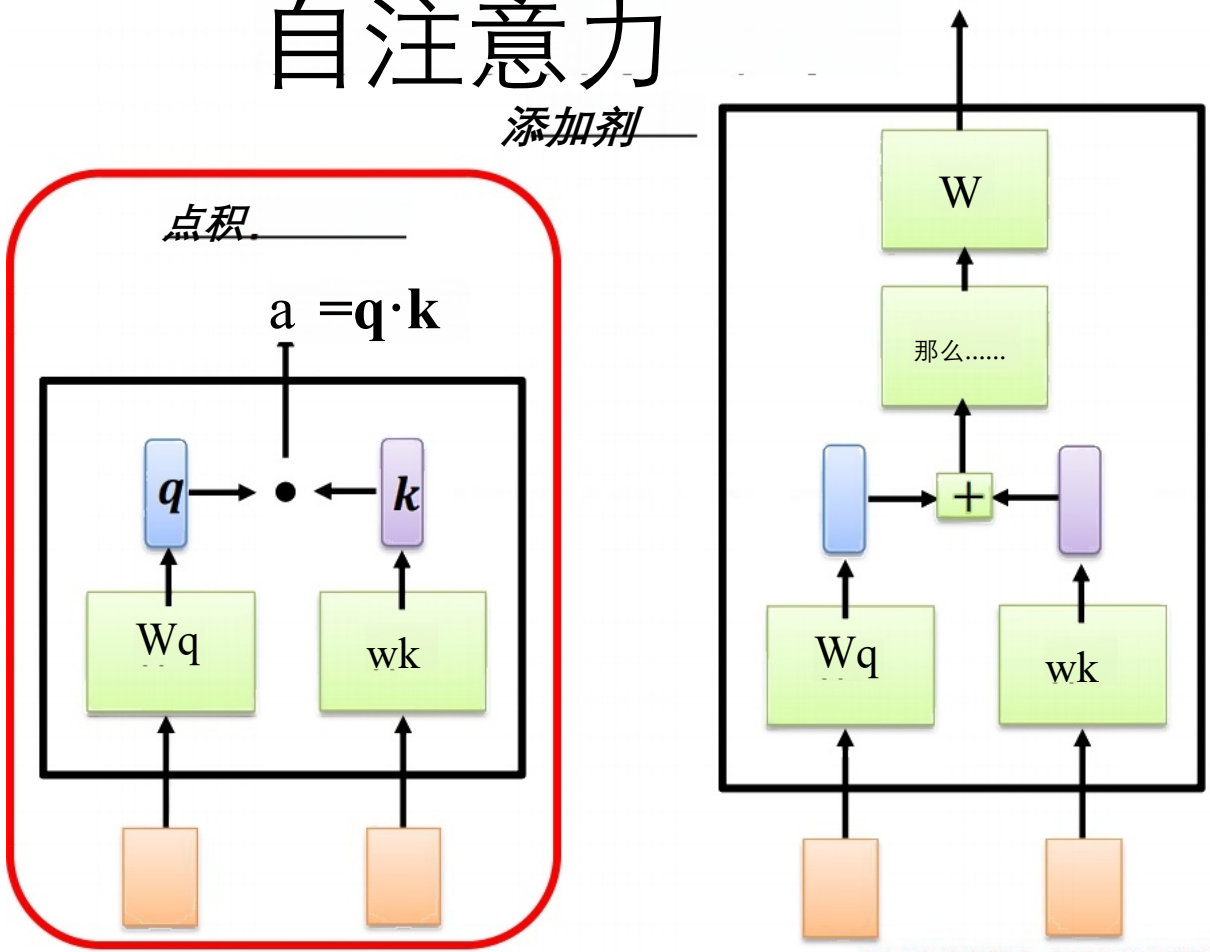
RENMIN 大学 OF 中国

自注意力

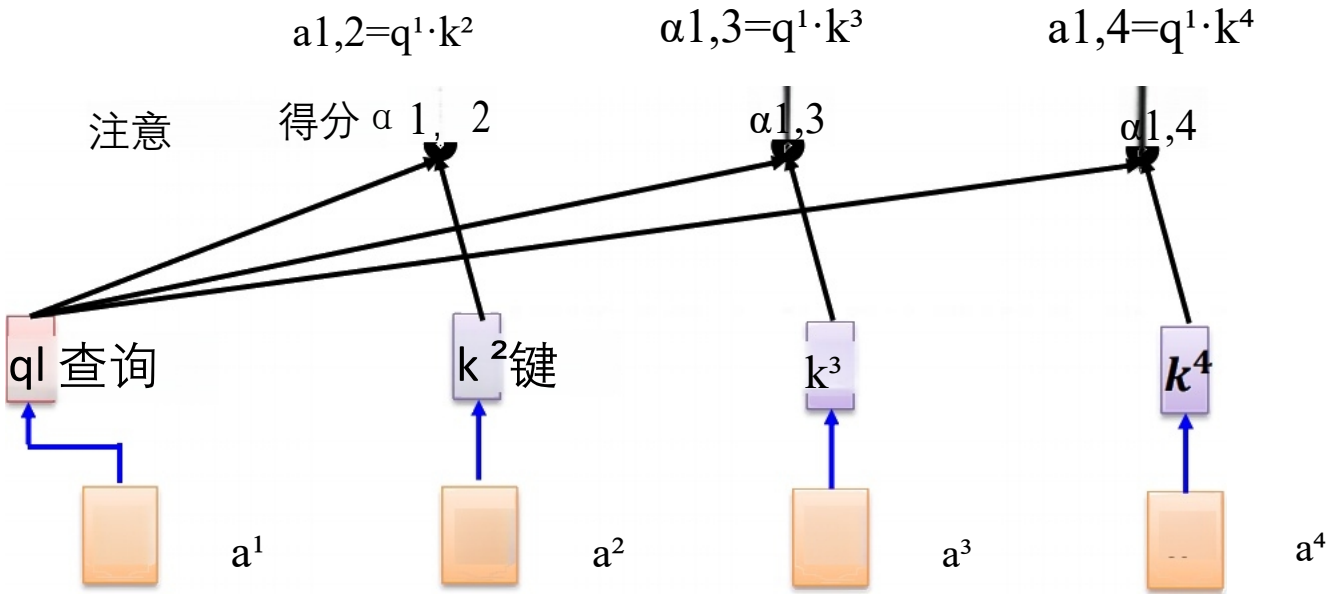


在序列中查找相关的向量

自注意力



自注意力

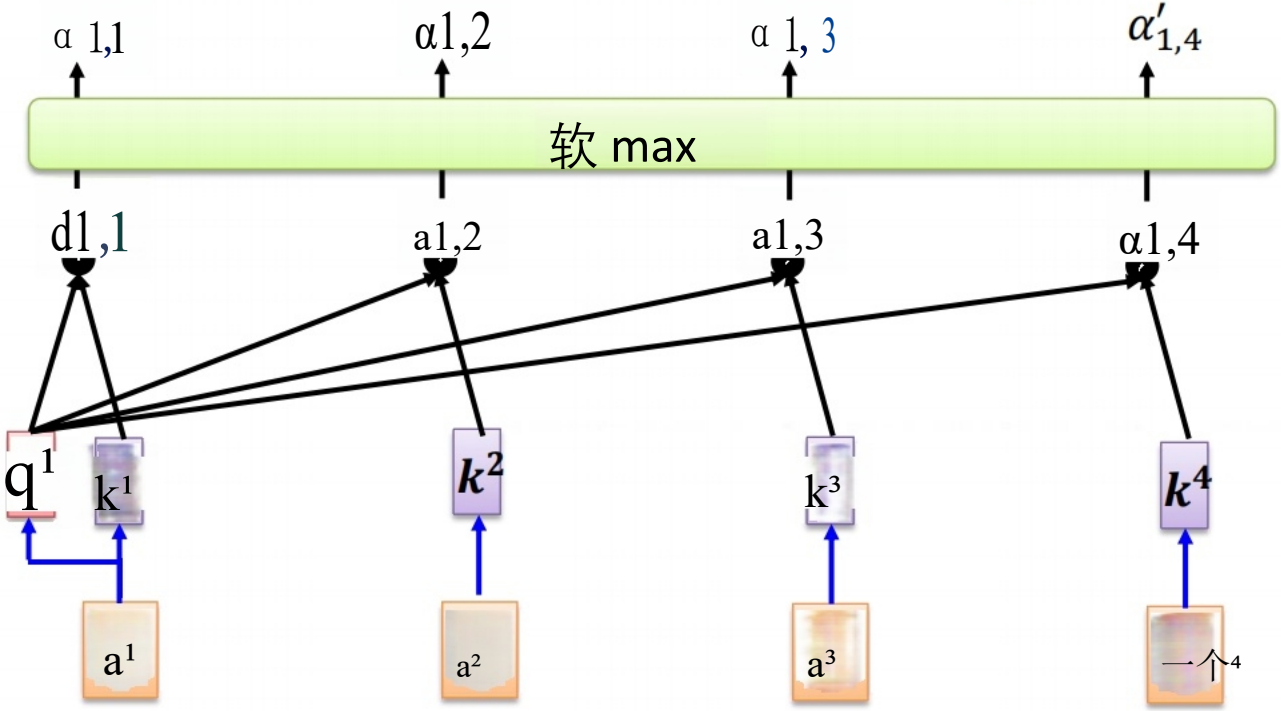


ql=错误$k^{2}=Wka^{2}$ k3=Wka 3 k $^{4}=Wka^{4}$

RENMIN 中国大学

自注意力

$$\alpha'_{1,i} = exp(\alpha_{1,i}) / \sum_j exp(\alpha_{1,j})$$



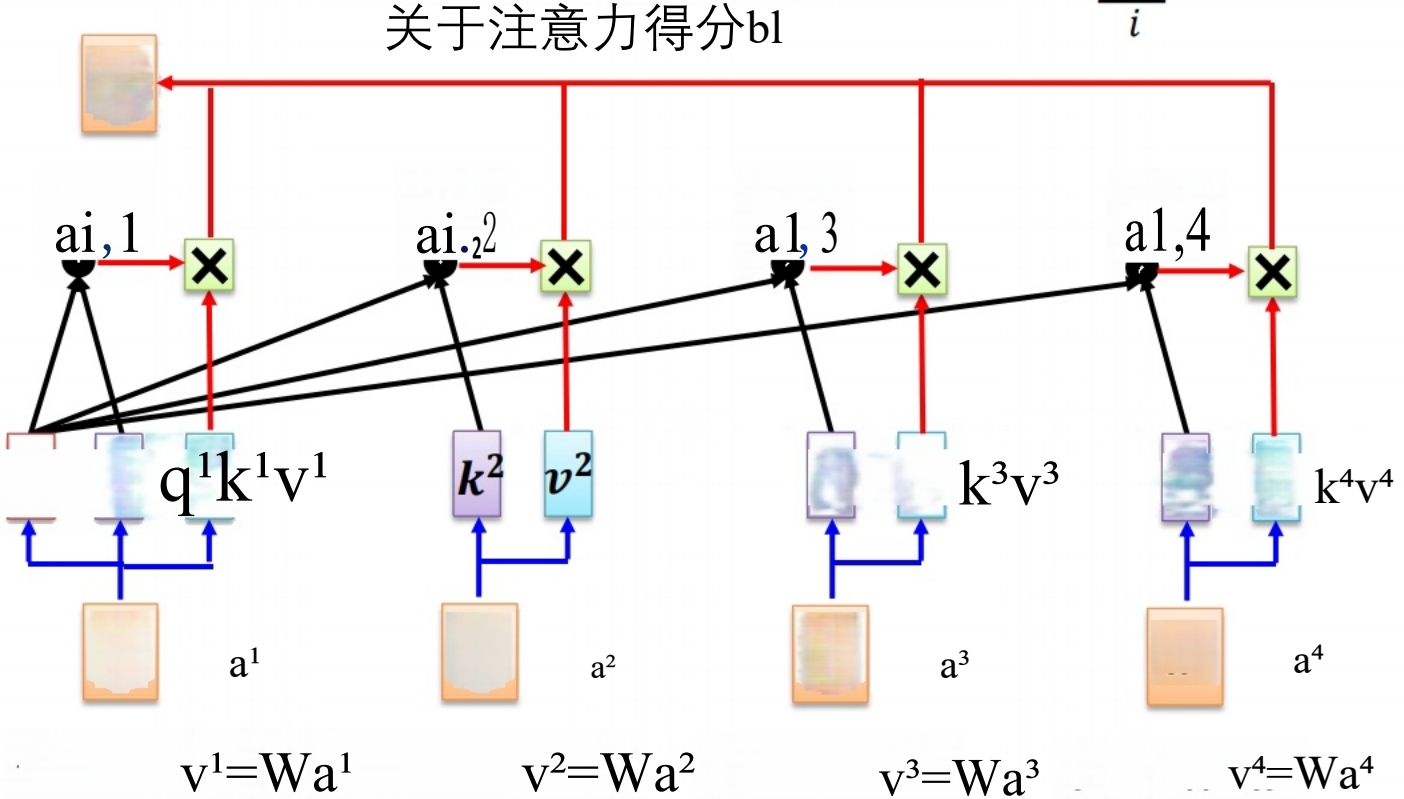
$q^1 = W^9 a^1$　　$k^2 = Wka^2$　　$k^3 = Wka^3$　　$k^4 = Wka^4$

$k^1 = Wka^1$

RENMIN 中国大学

自注意力：基于此提取信息

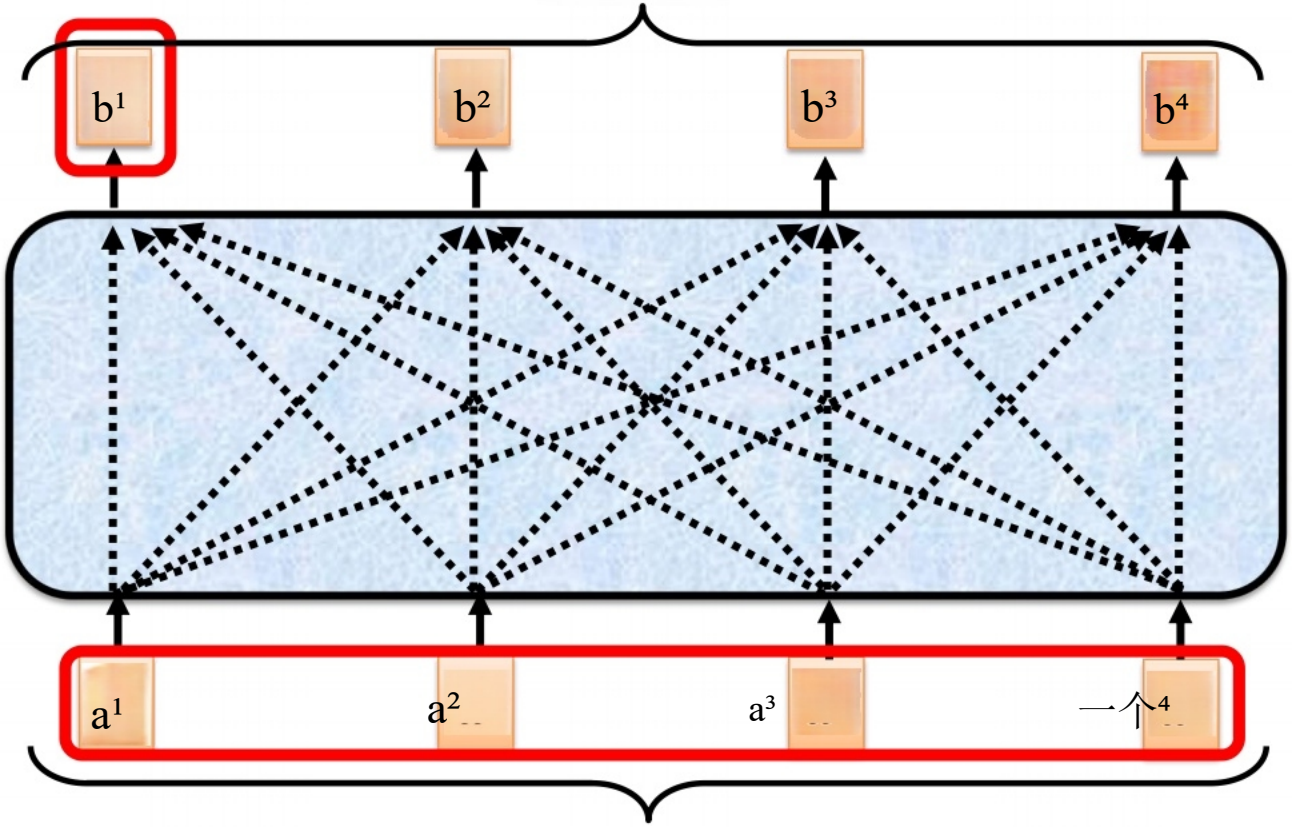
$$b^1 = \sum_i \alpha'_{1,i} v^i$$



RENMIN 中国大学

自注意力

平行的

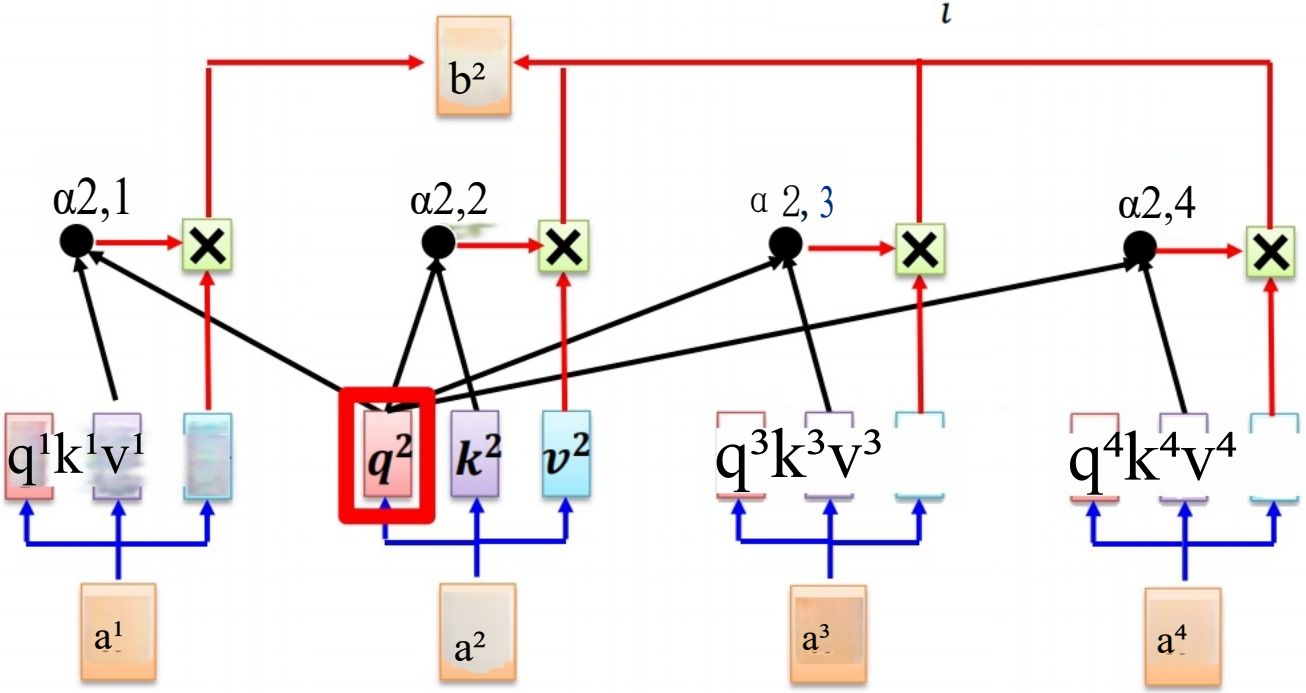


可以是输入层或隐藏层

RENMIN 中国大学

自注意力

$$b^2 = \sum_i \alpha'_{2,i} v^i$$



RENMIN 中国大学

自注意力——

$q^1 = Waa^2$

$q^1 q^2 q^3 q^4 = Wq \; a^1 a^2 a^3 a^4$

$Q \qquad\qquad I$

ki= *Wkai*

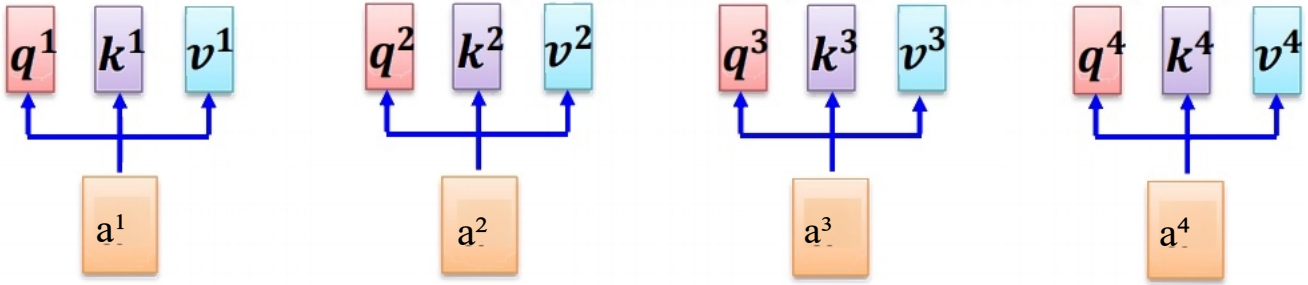
$k^1 k^2 k^3 k^4 = W^k \; a^1 a^2 a^3 a^4$

$K \qquad\qquad I$

$v^1 = Wa$

$v^1 v^2 v^3 v^4 = W \; a^1 a^2 a^3 a^4$

$V \qquad\qquad I$



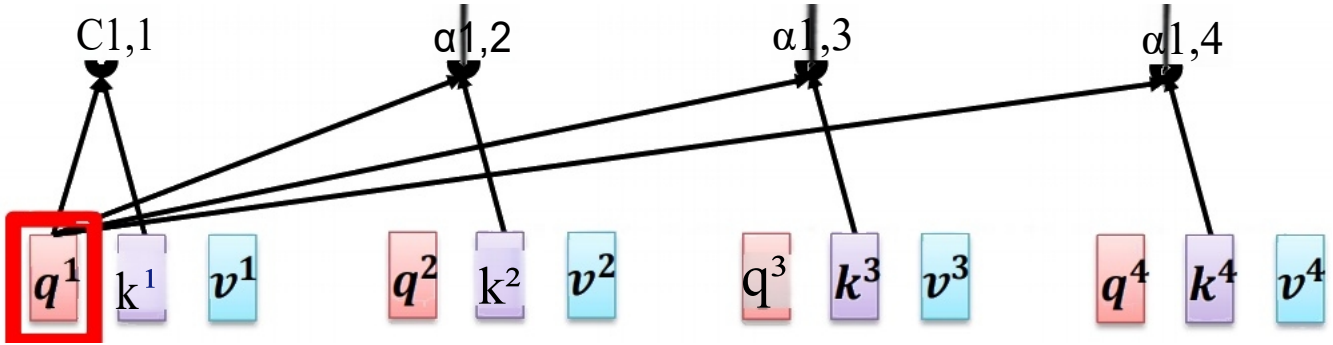
RENMIN 中国大学

自注意力

$\alpha_{1,1}=k^1 q^1$　　$\alpha_{1,2}=k^2 q^1$

$\alpha_{1,3}=k^3 q^1$　　$\alpha_{1,4}=k^4 q^1$

$\begin{matrix}\alpha_{1,1}\\ \alpha_{1,2}\\ \alpha_{1,3}\\ \alpha_{1,4}\end{matrix} = \begin{matrix}k^1\\ k^2\\ k^3\\ k^4\end{matrix} q^1$



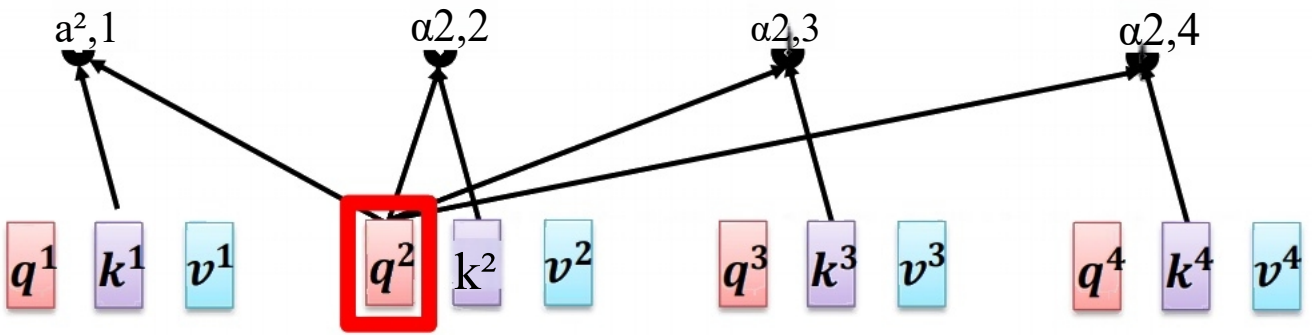
RENMIN 中国大学

自注意力

$\alpha_{1,1} = k^1 q^1$　　$\alpha_{1,2} = k^2 q^1$

$\alpha_{1,3} = k^3 q^1$　　$\alpha_{1,4} = k^4 q^1$

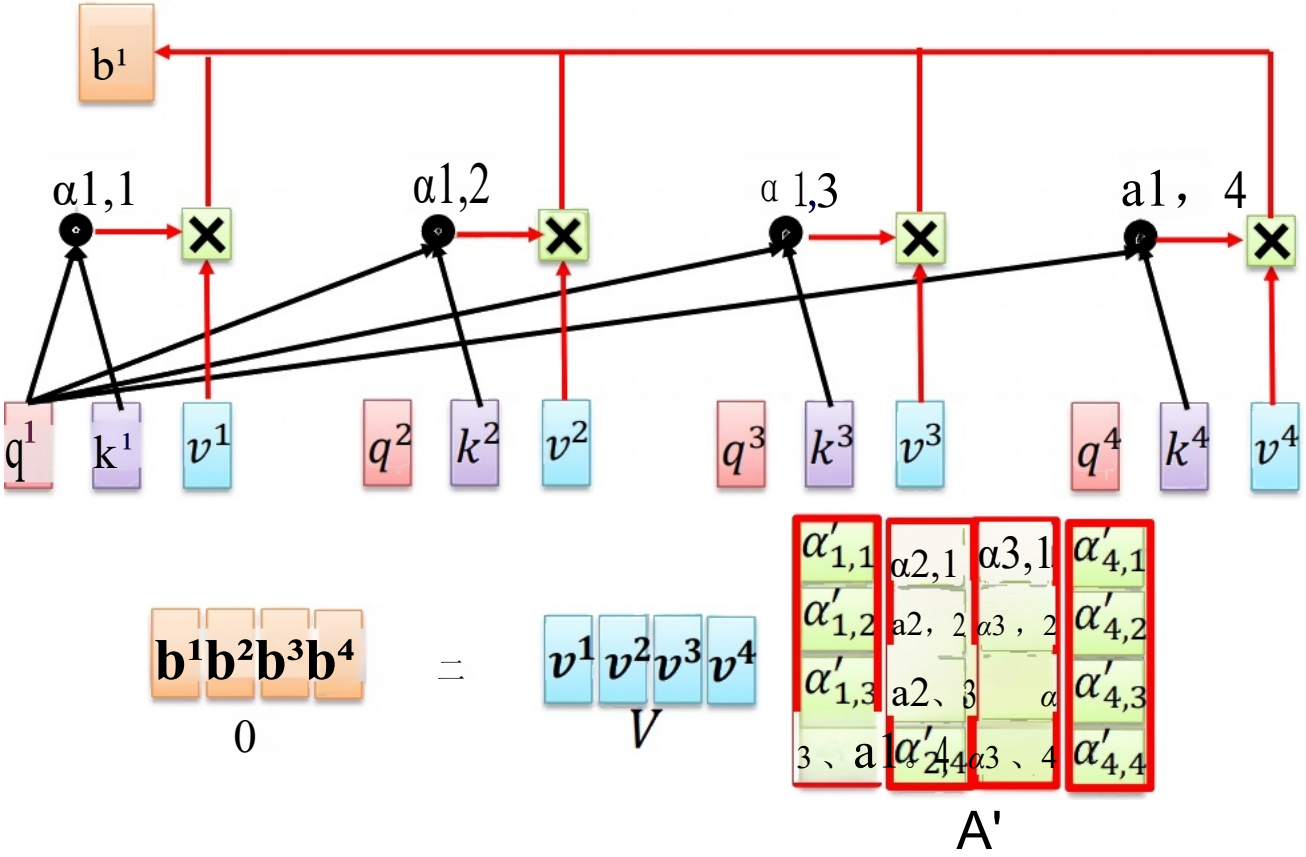
$$\begin{bmatrix}\alpha_{1,1}\\ \alpha_{1,2}\\ \alpha_{1,3}\\ \alpha_{1,4}\end{bmatrix} = \begin{bmatrix}k^1\\ k^2\\ k^3\\ k^4\end{bmatrix} q^1$$



$$\begin{bmatrix}\alpha'_{1,1} & \alpha'_{2,1} & \alpha'_{3,1} & \alpha_{4,1}\\ \alpha'_{1,2} & \alpha'_{2,2} & \alpha_{3,2} & \alpha_{4,2}\\ \alpha'_{1,3} & \alpha'_{2,3} & \alpha_{3,3} & \alpha_{4,3}\\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4}\end{bmatrix} A' \quad \leftarrow \quad \begin{bmatrix}\alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1}\\ \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2}\\ \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3}\\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4}\end{bmatrix} A = \begin{bmatrix}k^1\\ k^2\\ k^3\\ k^4\end{bmatrix} K^T \begin{bmatrix}q^1 & q^2 & q^3 & q^4\end{bmatrix} Q$$

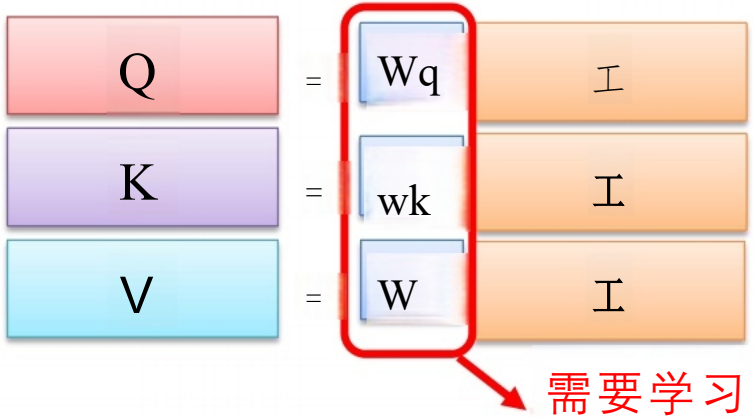
RENMIN 中国大学

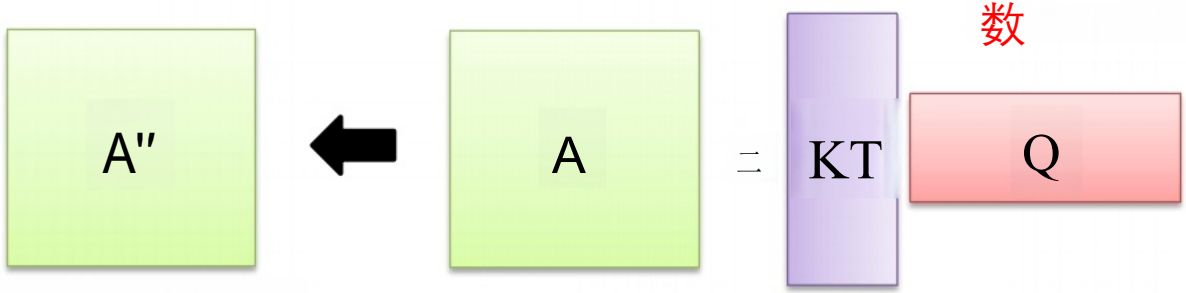
自注意力



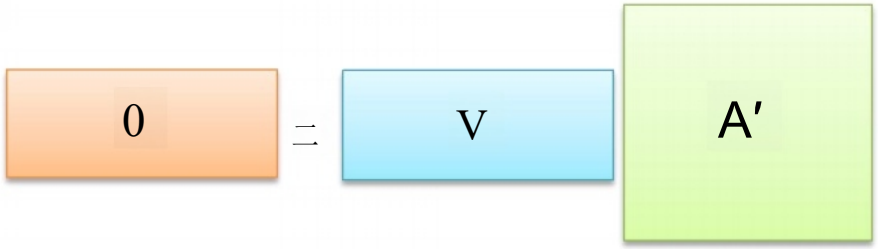
RENMIN 中国大学

自注意力



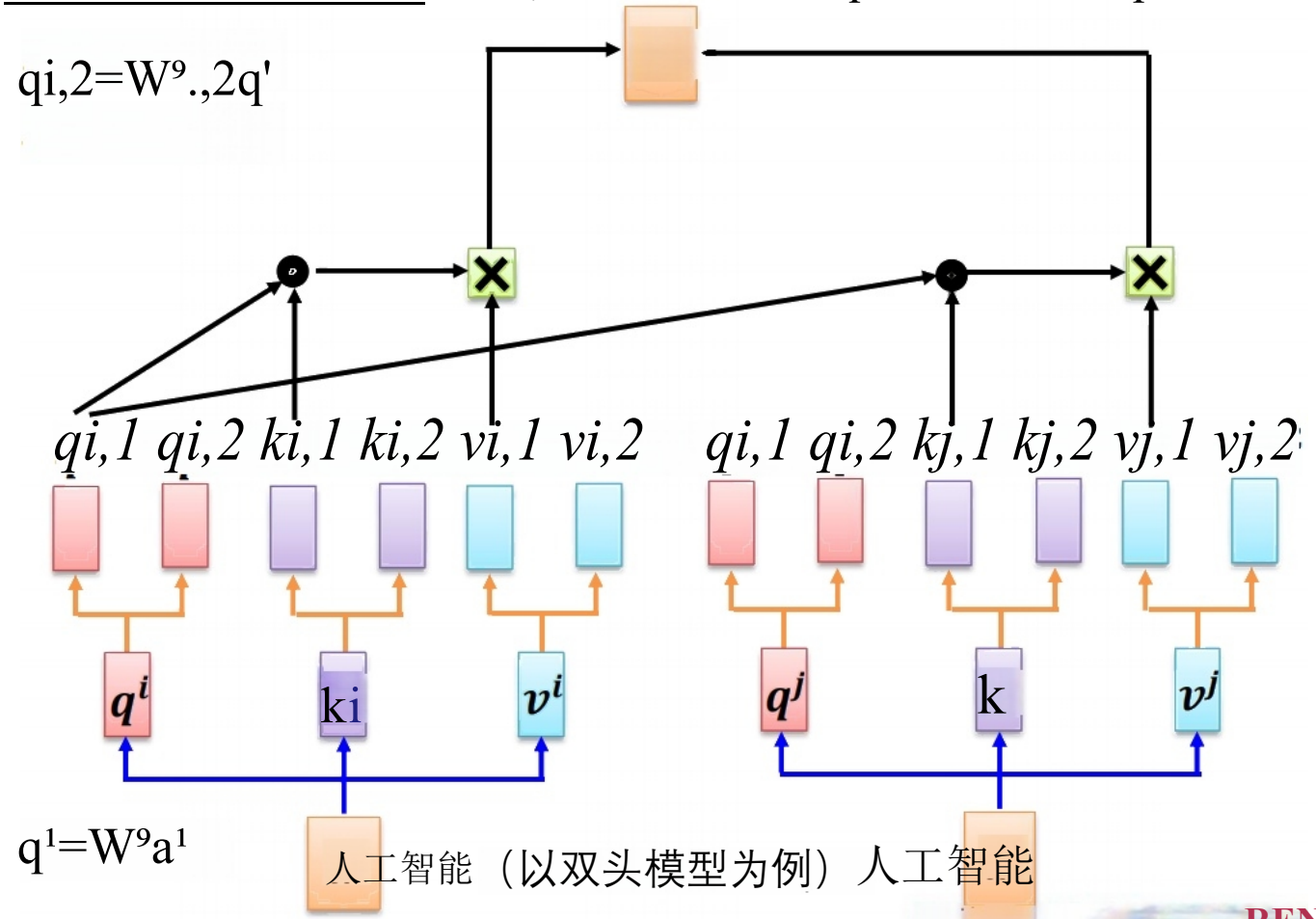


注意；关心 矩阵



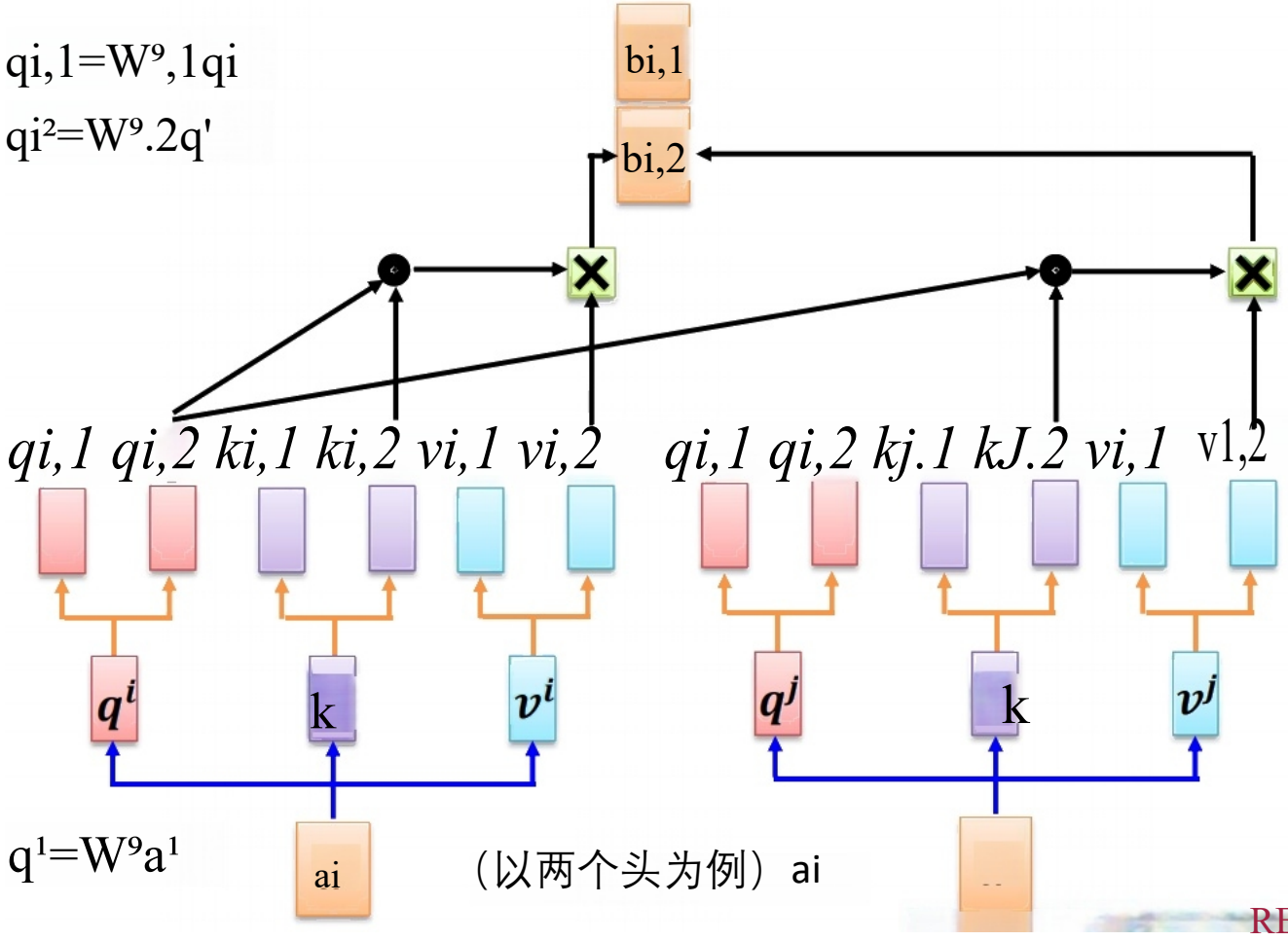
RENMIN 中国大学

多头自注意力机制不同类型的关联度qi，1=W⁹，1qi bi,1←



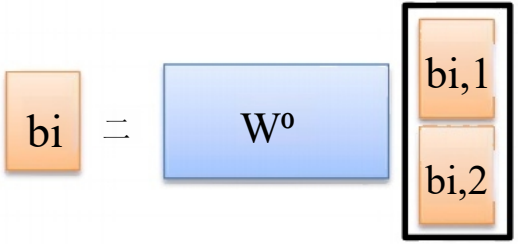
RENMIN 中国大学

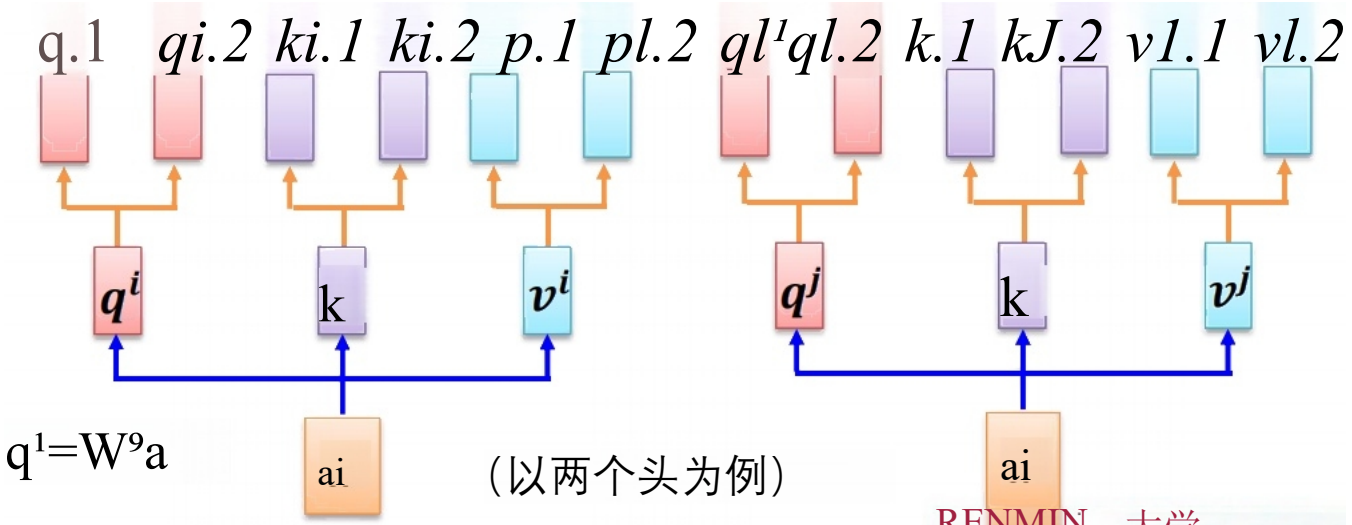
多头自注意力机制不同类型的关联性



RENMIN 中国大学

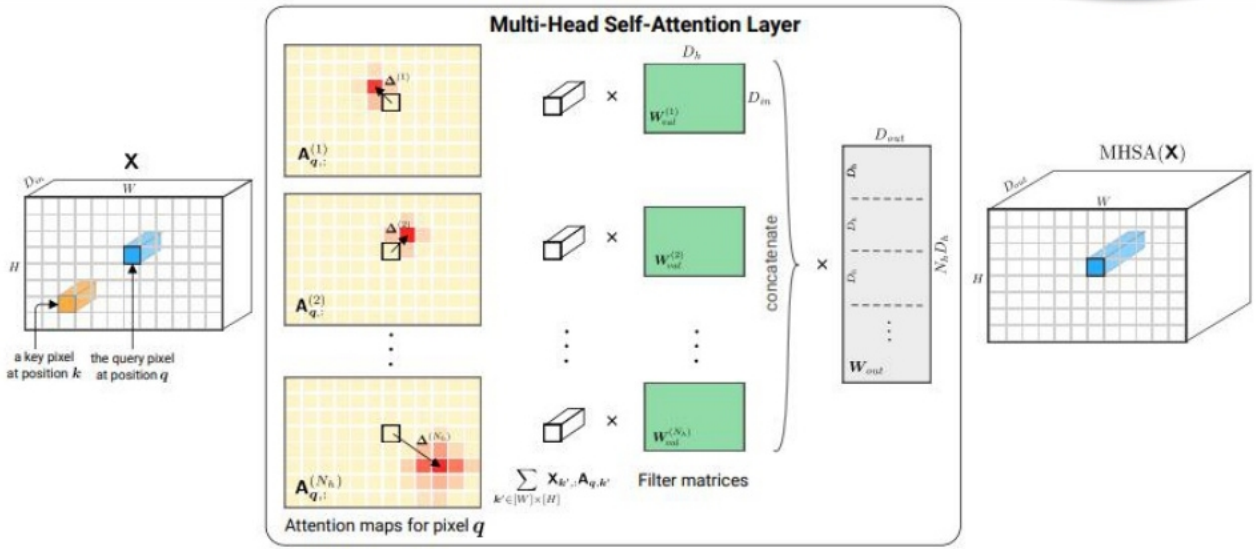
多头自注意力机制不同类型的关联性





自注意力机制与卷积神经网络（CNN）的对比





关于自注意力与卷积层之间的关系https://arxiv.org/abs/1911.03584

Transformer

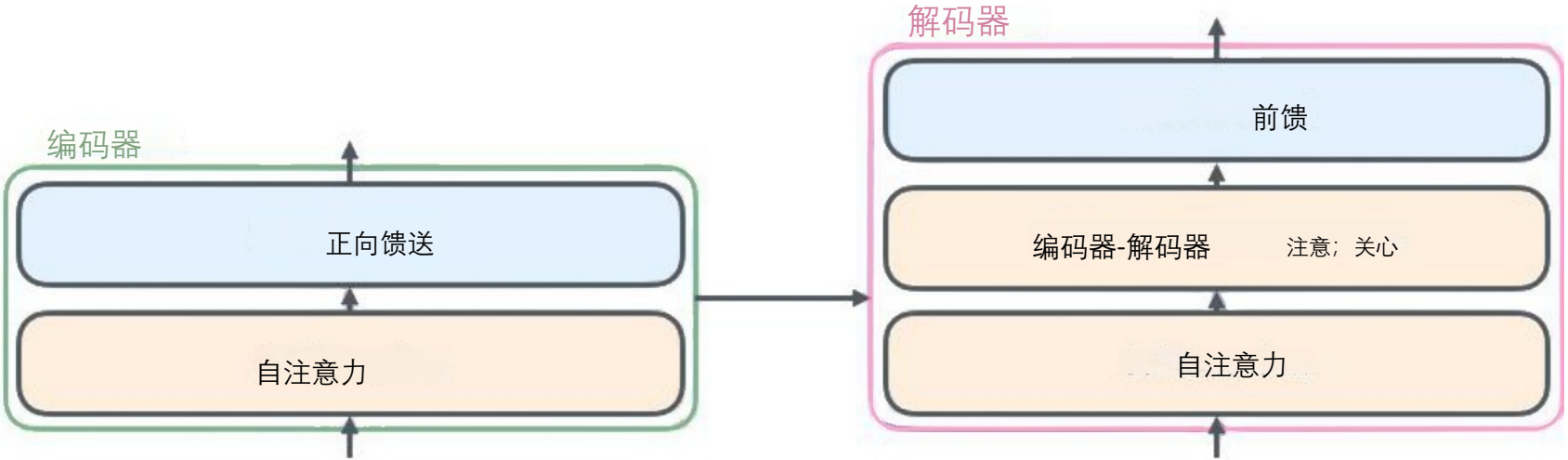
- Transformer是一种用于机器翻译任务的模型架构，它通过注意力机制在多项任务中超越了先前采用注意力机制的序列到序列 SOTA 。

- Transformer是一种序列到序列的模型架构。其区别在于采用了改进版的注意力机制——自注意力机制；该机制不仅具有强大的表达能力，还具备可表示为矩阵运算这一计算优势。

- 核心架构由一系列编码器与一系列解码器完全连接而成。每个编码器包含两个模块：一个自注意力组件和一个前馈网络；每个解码器则包含三个模块：一个自注意力组件、一个编码器-解码器注意力组件以及一个前馈组件。

RENMIN 中国大学

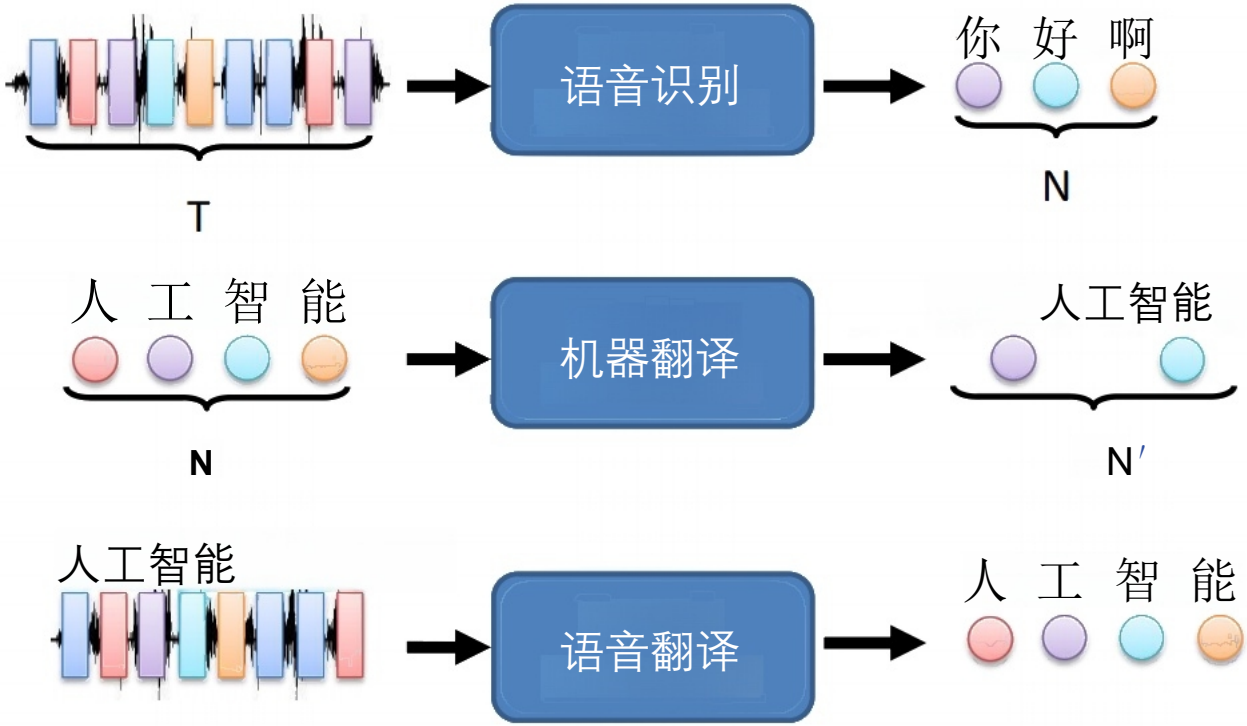
Transformer



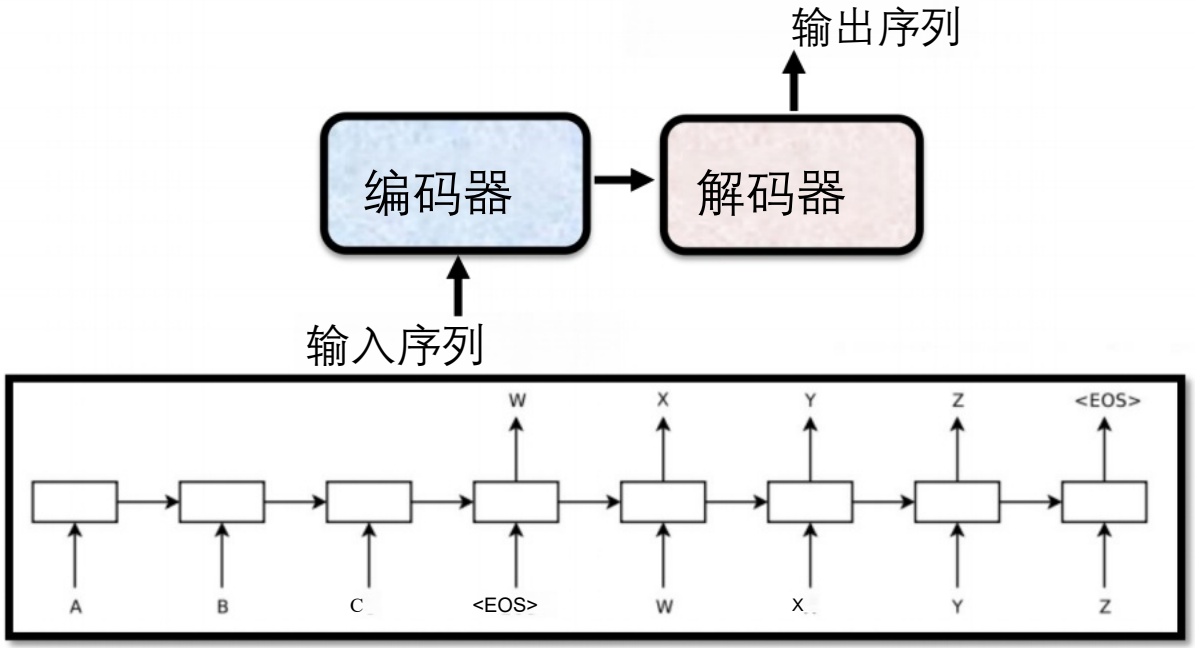
序列到序列（Seq2seq）

输入一个序列，输出一个序列

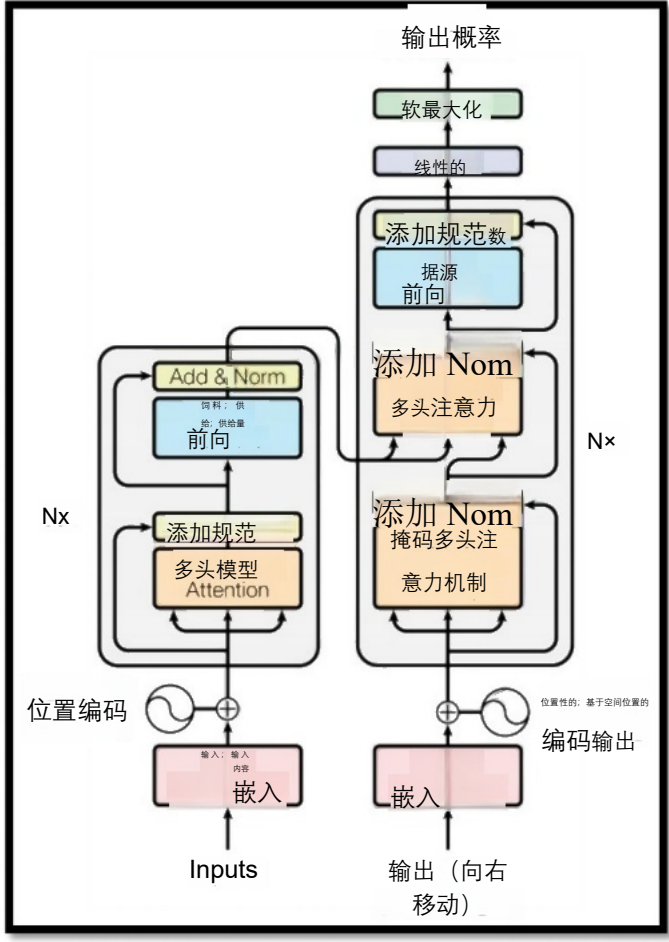
输出长度由模型决定。



Seq2seq



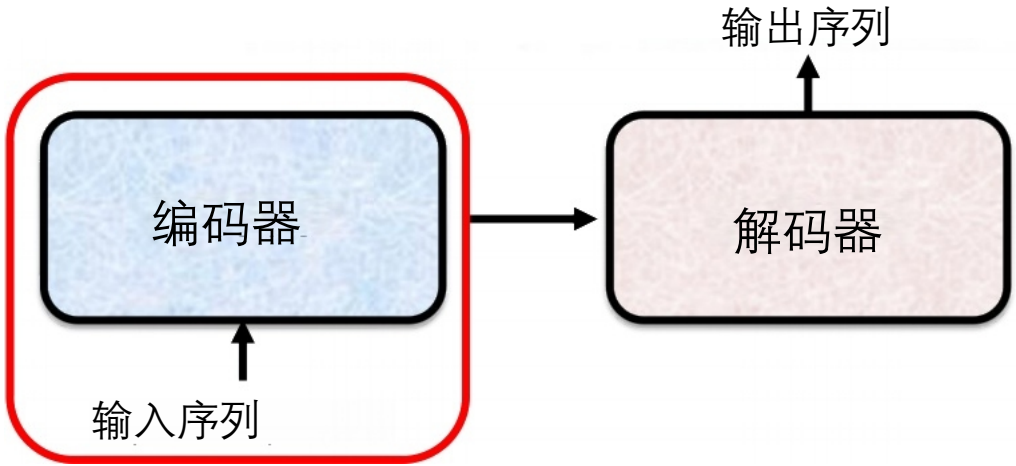
基于神经网络的序列到序列学习https://arxiv.org/abs/1409.3215



Transformer
https://arxiv.org/abs/1706.03762

RENMIN 中国大学

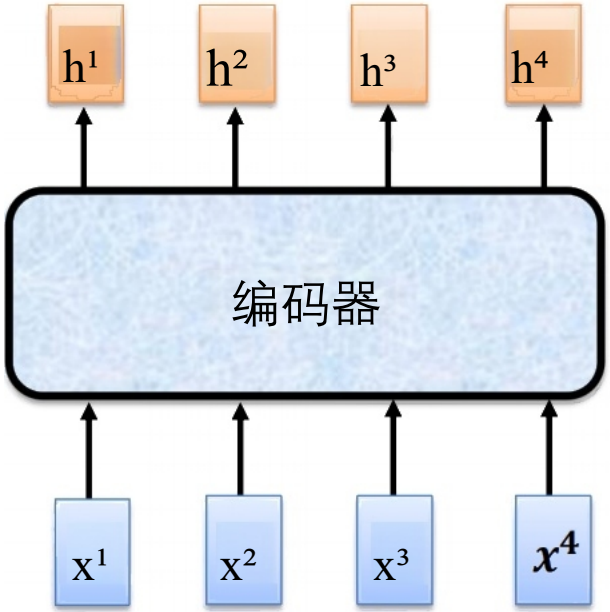
编码器



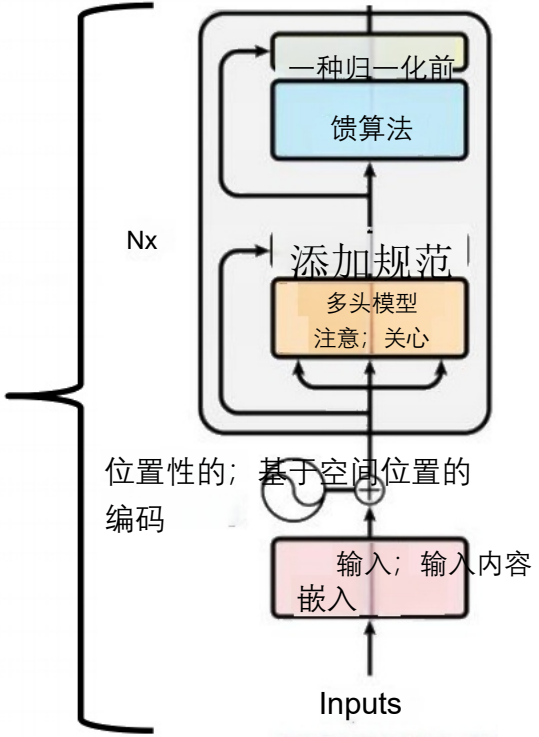
RENMIN 中国大学

编码器

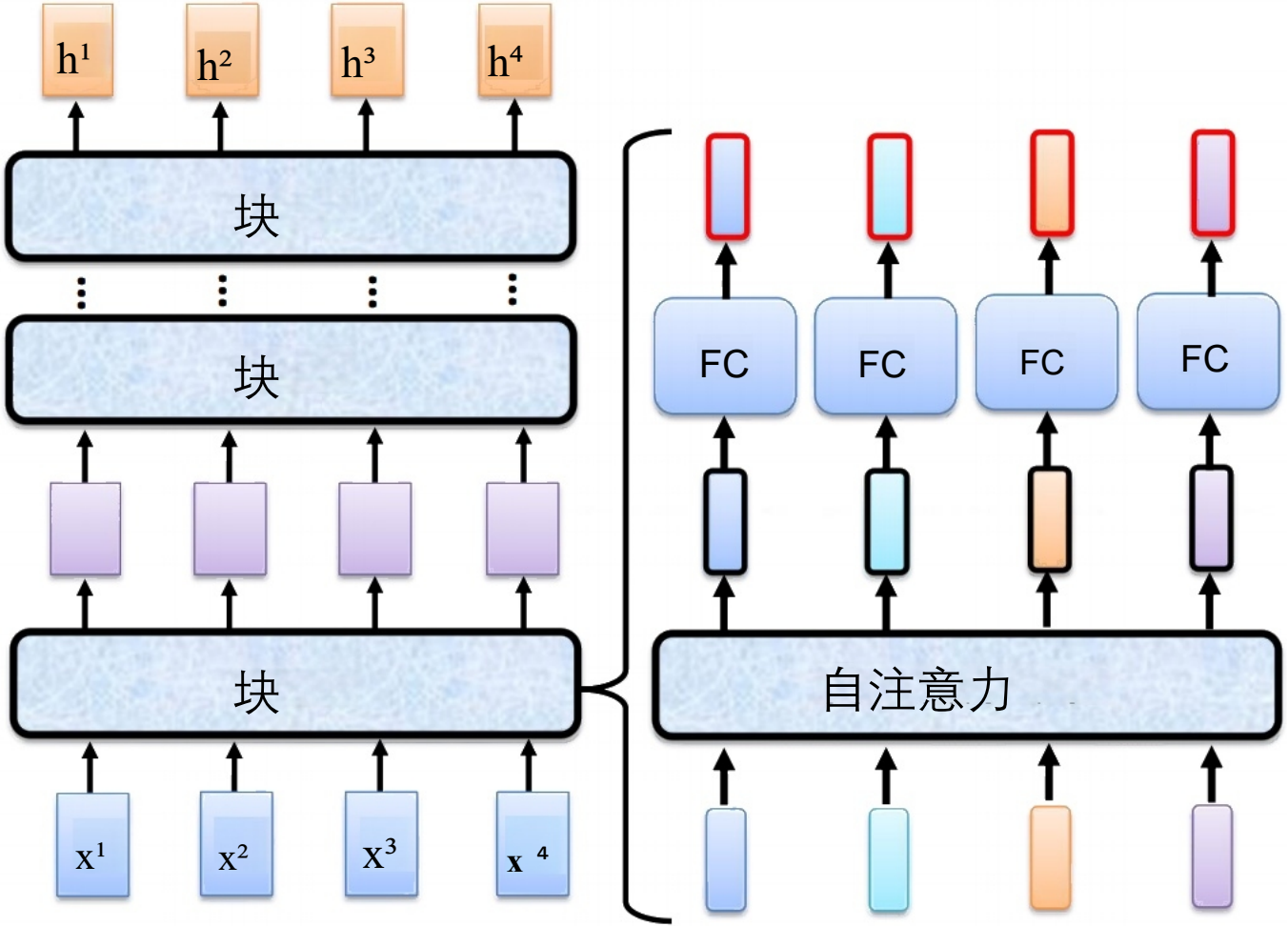
使用**RNN**或**CNN**。



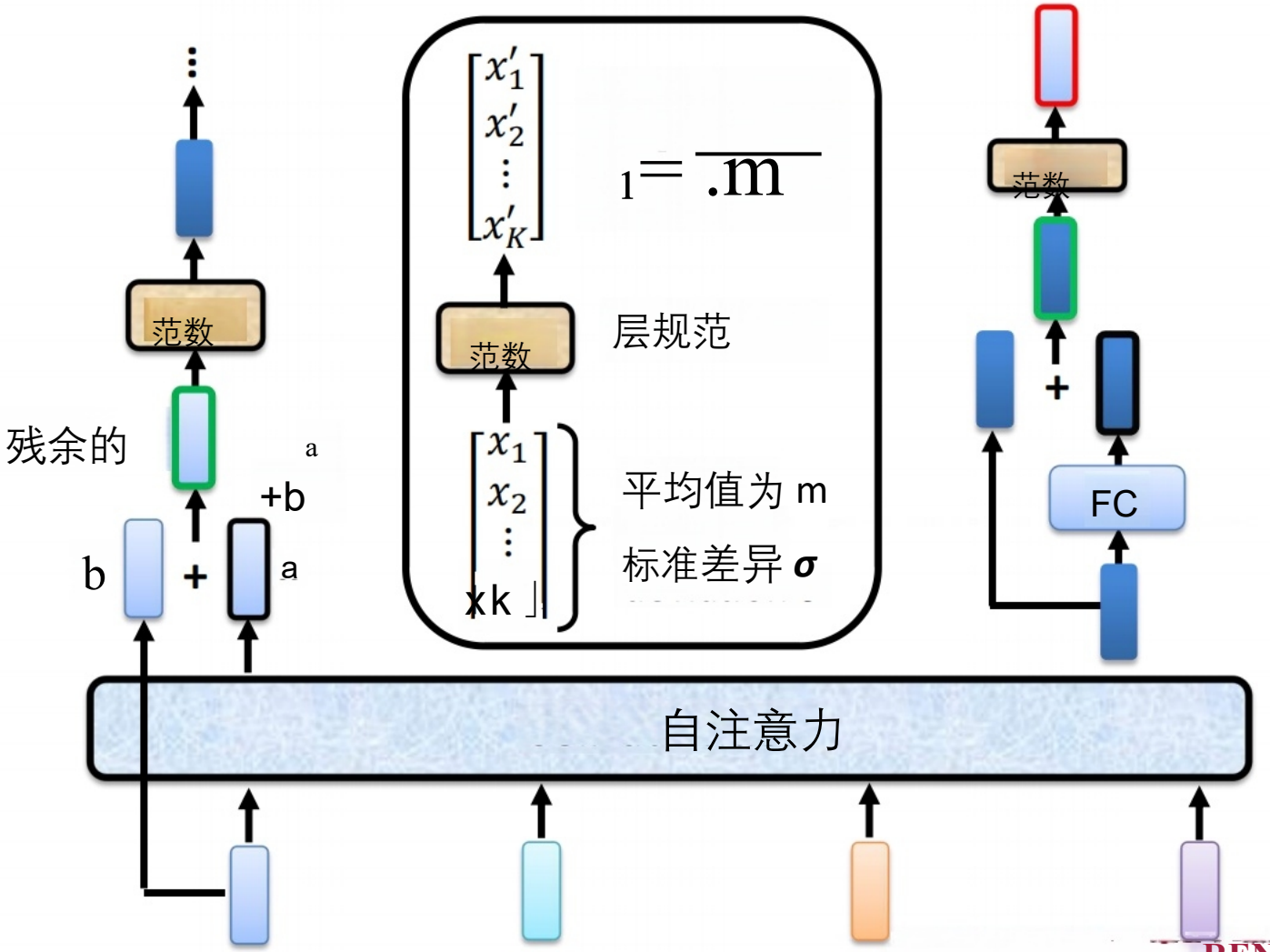
Transformer的编码器您可以



RENMIN 中国大学



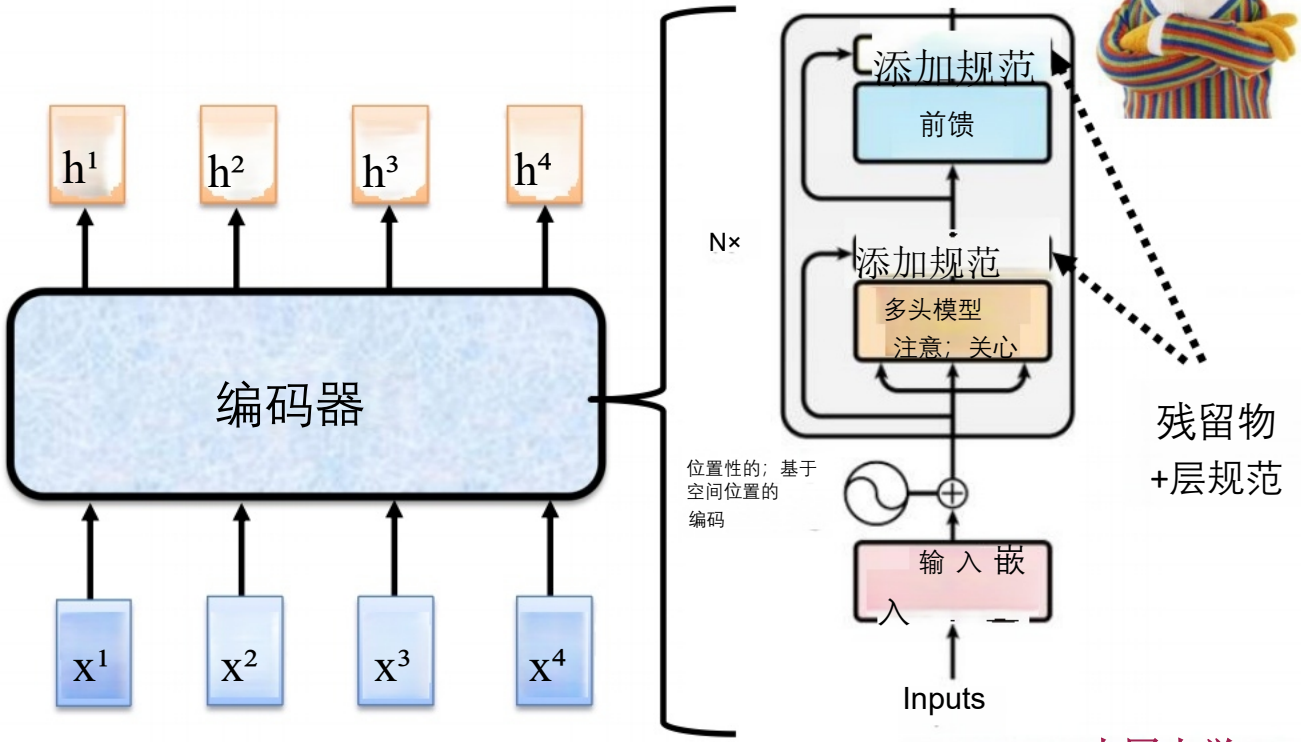
RENMIN 中国大学


范数
残余的
a
+b
b
+
a
自注意力
范数
层规范
平均值为 m
标准差异 σ
范数
+
FC

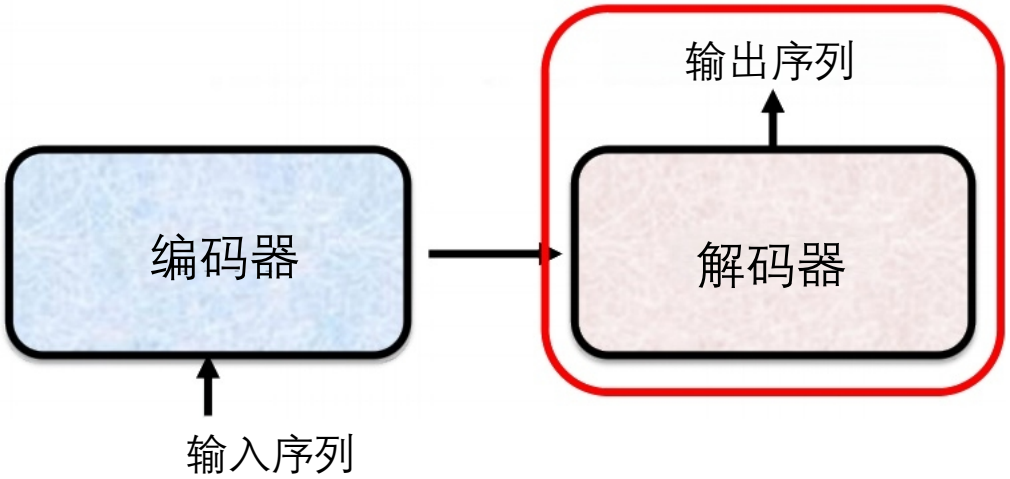
BERT

我采用了与Transformer编码器相同的网络架构。



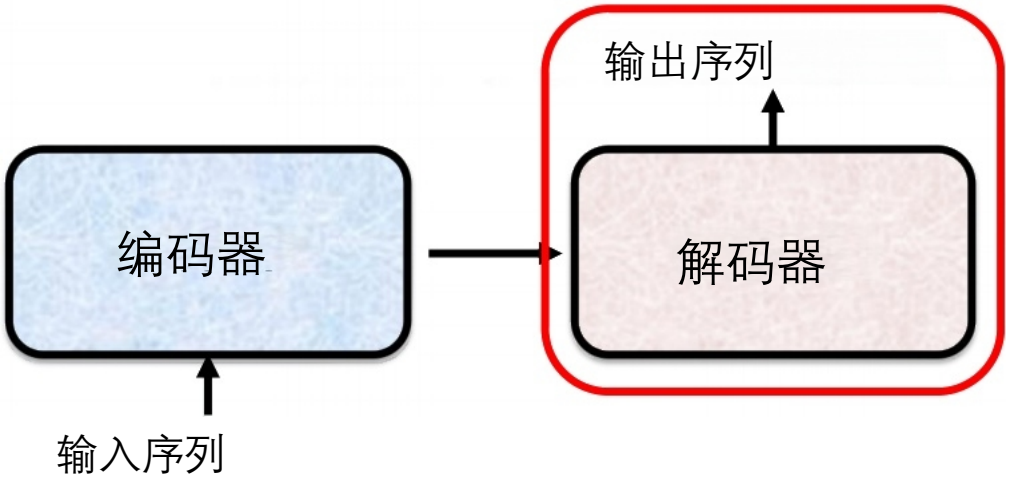


解码器



RENMIN 中国大学

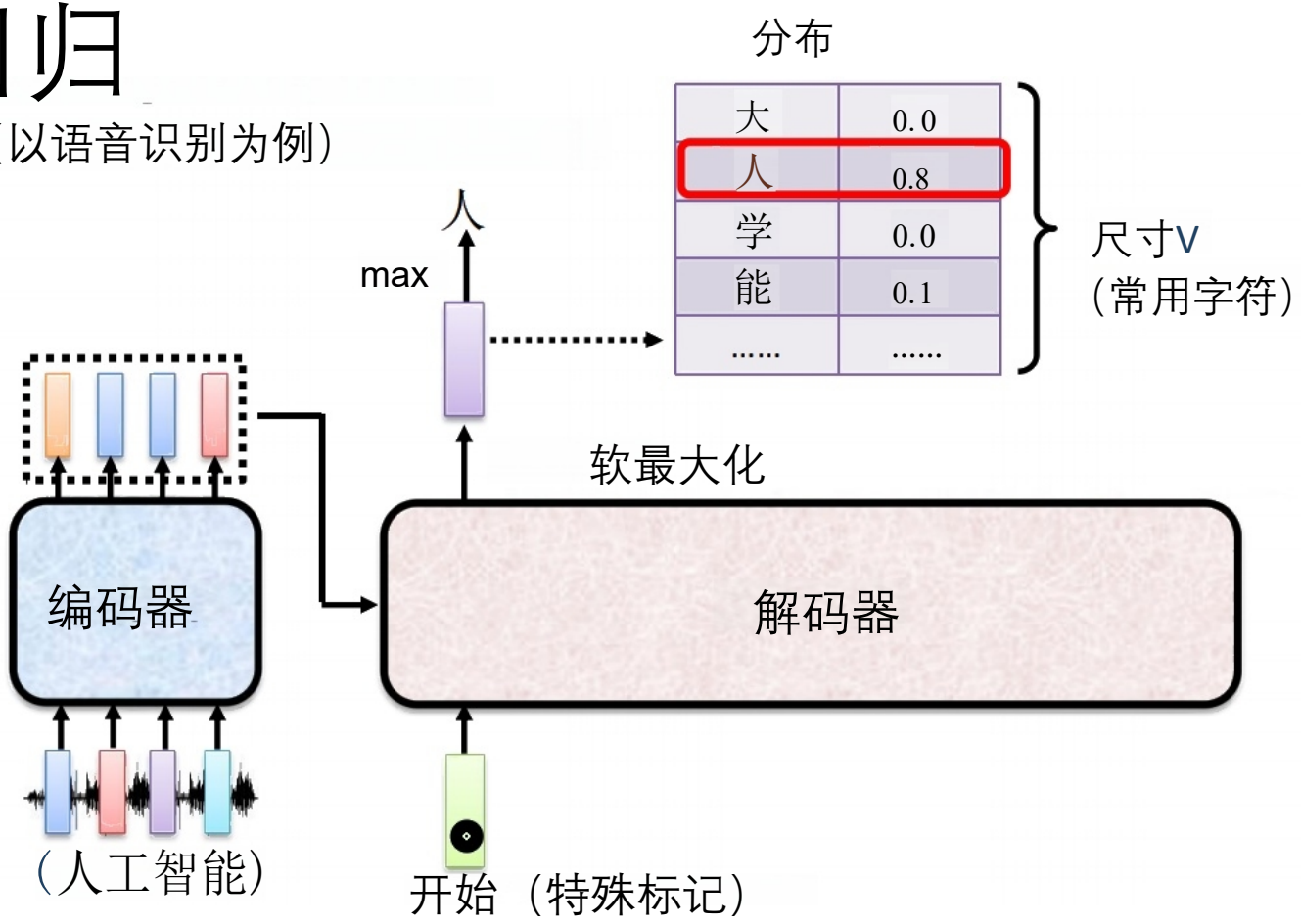
解码器
-自回归（AT）



RENMIN 中国大学

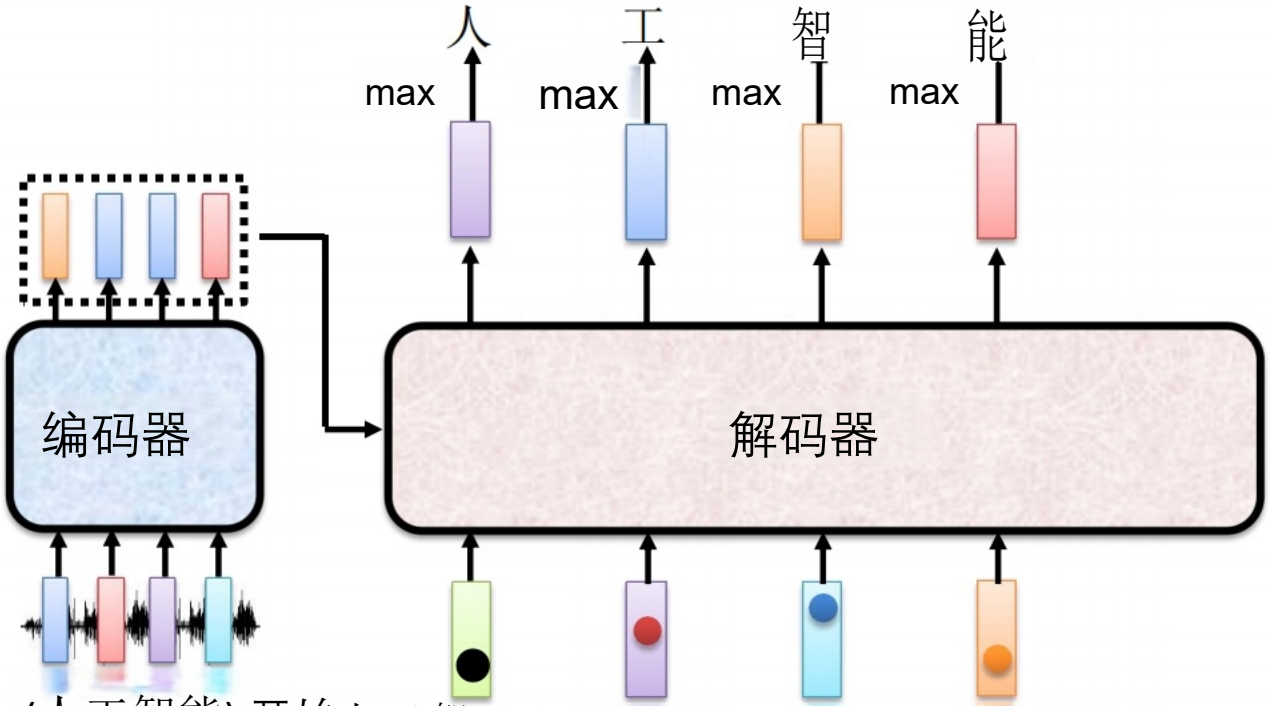
自回归

（以语音识别为例）

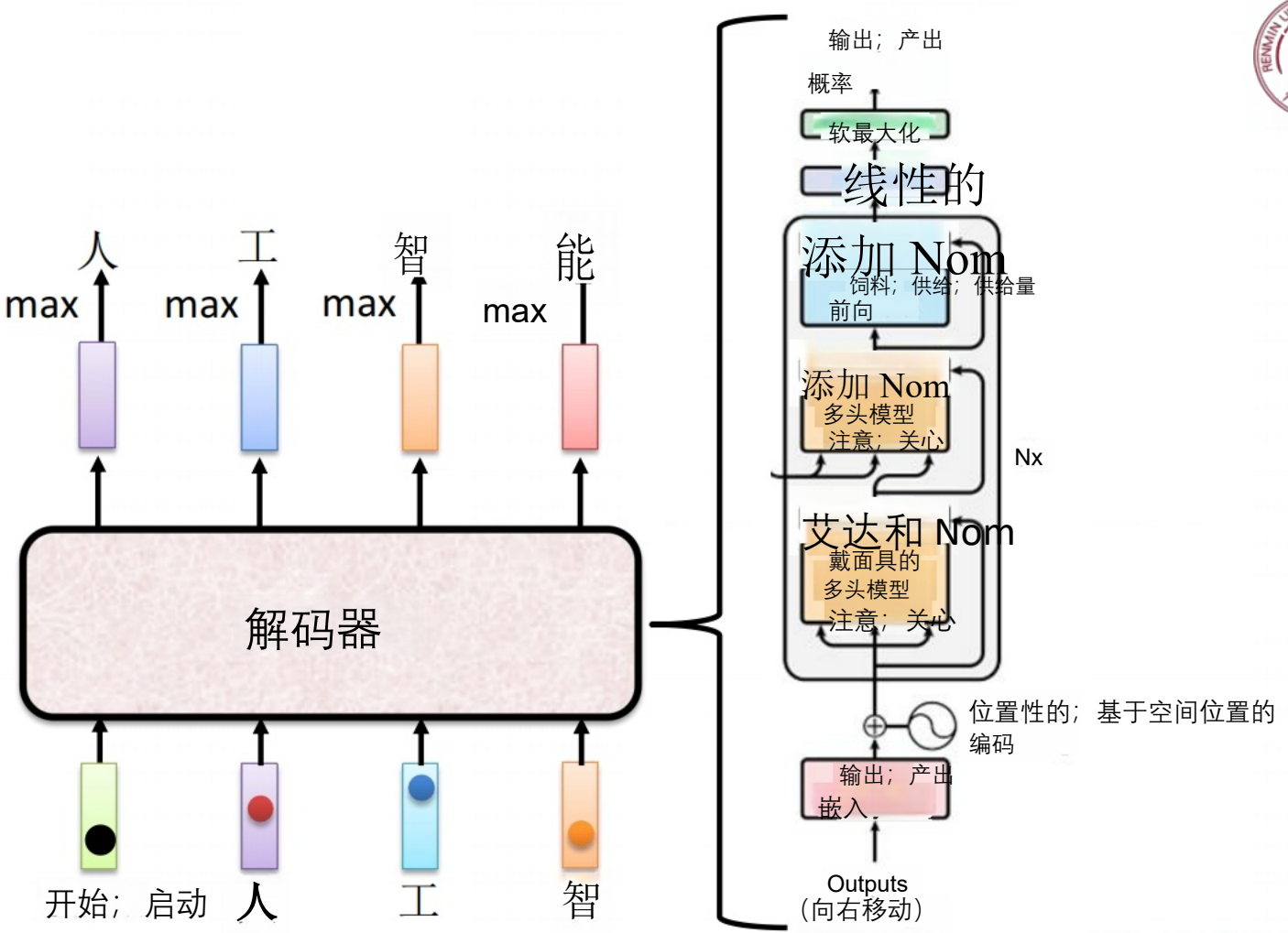


RENMIN 中国大学

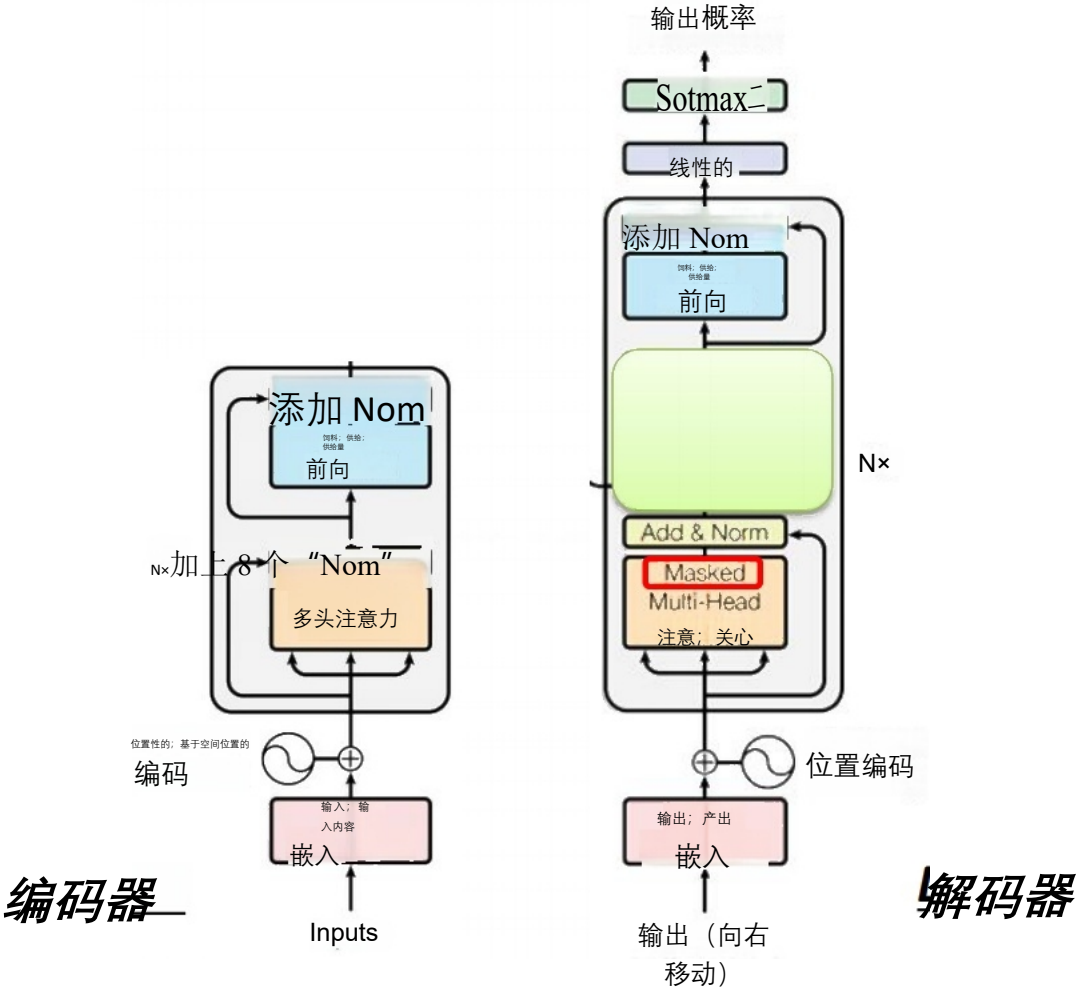
自回归



RENMIN 中国大学

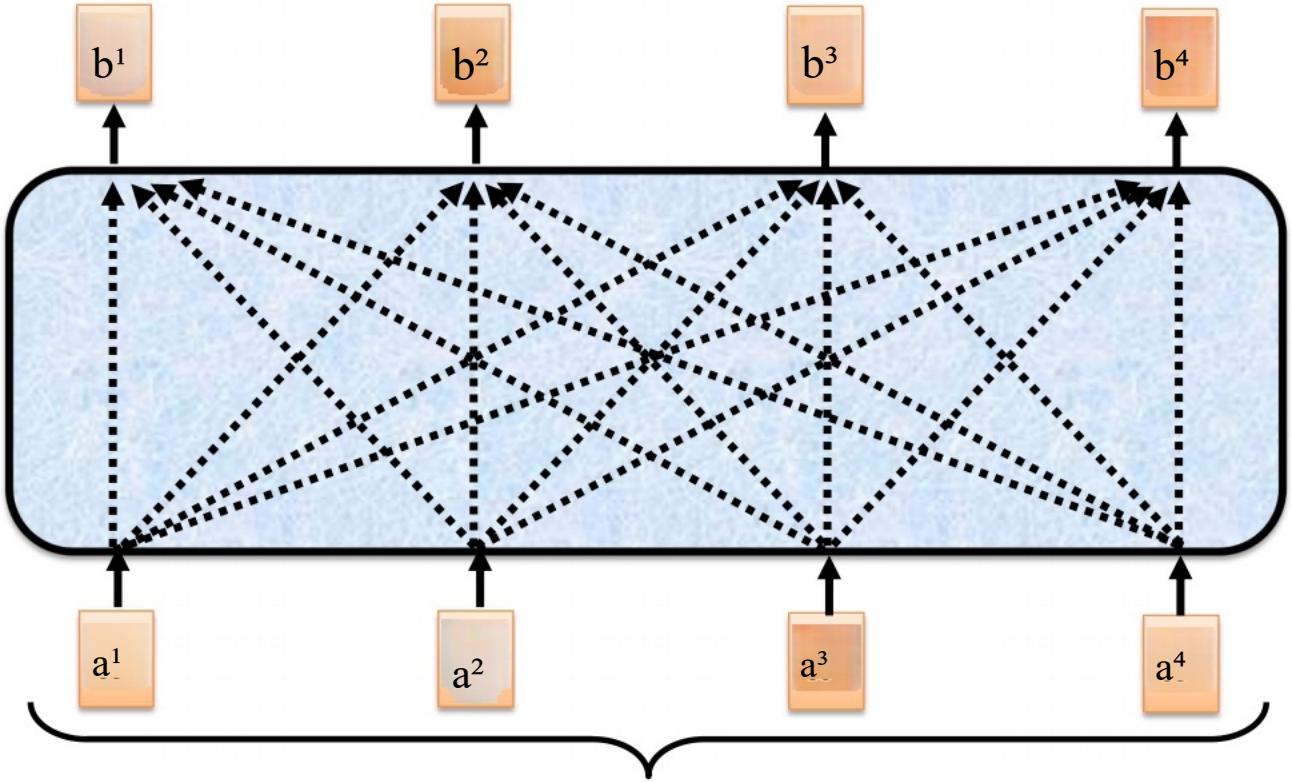

人
工
智
能
max
max
max
max
解码器
开始；启动
人
工
智
输出；产出
概率
软最大化
线性的
添加 Nom
饲料；供给；供给量
前向
添加 Nom
多头模型
注意；关心
艾达和 Nom
戴面具的
多头模型
注意；关心
Nx
位置性的；基于空间位置的
编码
输出；产出
嵌入
Outputs
（向右移动）

RENMIN 中国大学


输出概率
Sotmax
线性的
添加 Nom
前向
N×
Add & Norm
Masked
Multi-Head
注意；关心
位置编码
输出；产出
嵌入
输出（向右移动）
解码器
添加 Nom
前向
N×加上 8 个 “Nom”
多头注意力
位置性的；基于空间位置的
编码
输入；输入内容
嵌入
Inputs
编码器

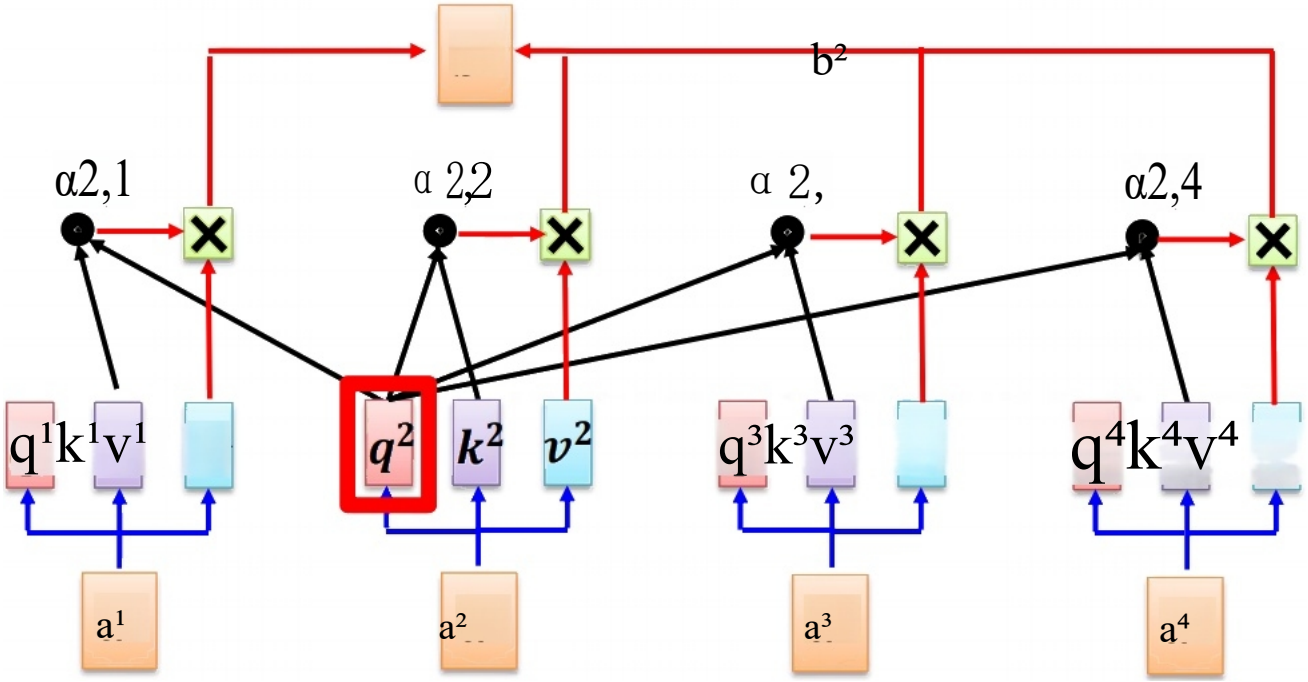
RENMIN 中国大学

~~自注意力→~~ ~~掩蔽自注意力~~



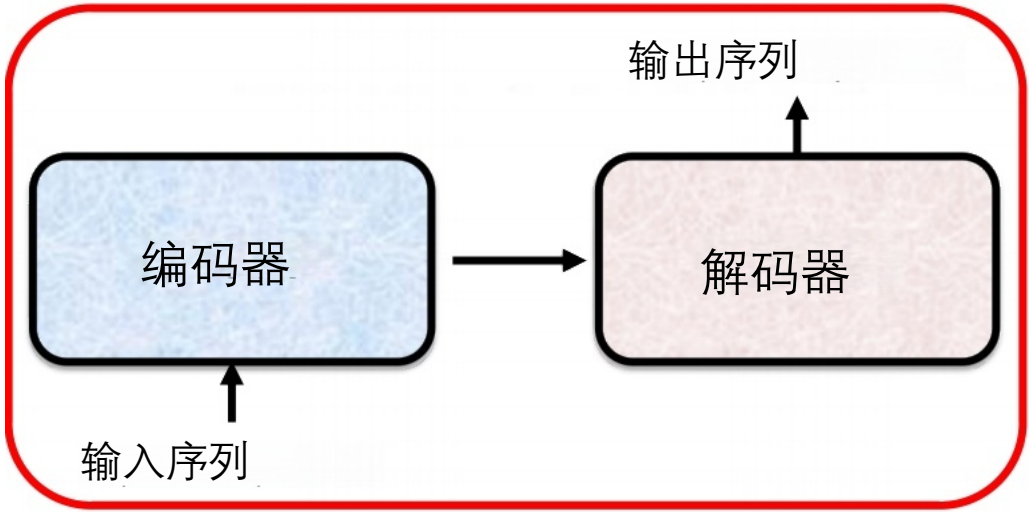
可以是输入层或中间隐藏层

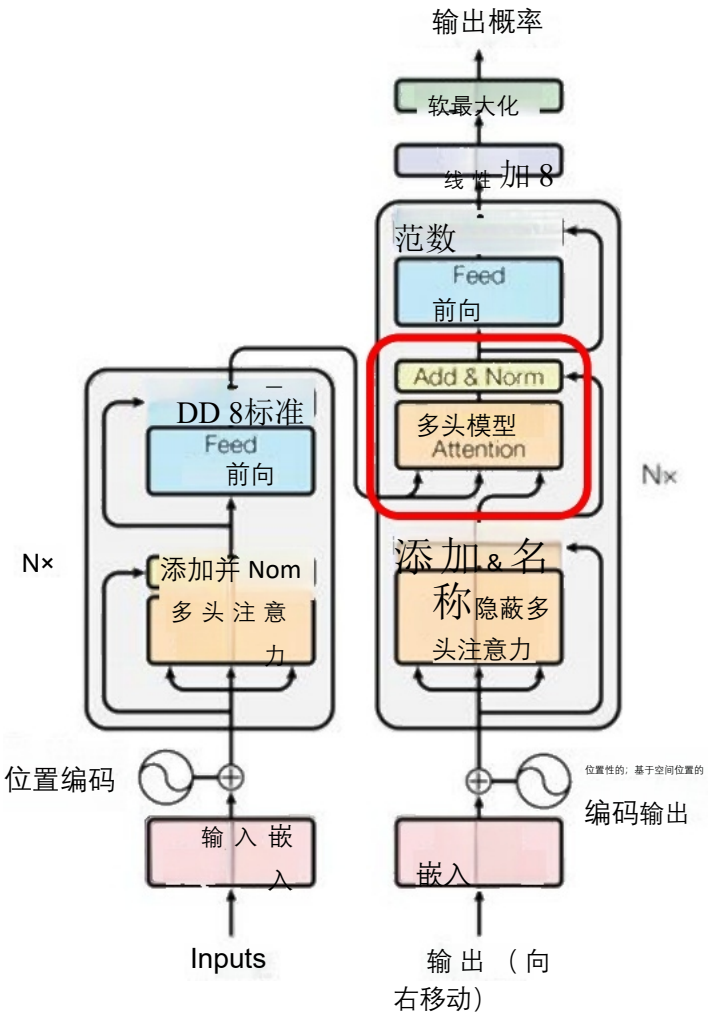
自注意力→ 掩蔽自注意力



为什么要使用掩码？请考虑解码器的工作原理。

编码器-解码器




输出概率
软最大化
线性
加 8
范数
Feed
前向
Add & Norm
多头模型
Attention
N×
DD 8标准
Feed
前向
添加并 Nom
多头注意力
N×
添加& 名称
隐蔽多头注意力
位置编码
输入嵌入
嵌入
位置性的；基于空间位置的
编码输出
Inputs
输出（向右移动）

使用 BERT 对文本数据进行分类

- 本示例展示了如何使用预训练的BERT模型作为特征提取器对文本数据进行分类。
- 预训练BERT模型最简单的用途是将其作为特征提取器。具体而言，可利用该模型将文档转换为特征向量，进而作为输入数据训练深度学习分类网络。
- 本示例展示了如何利用预训练的BERT模型，基于工厂报告数据集对故障事件进行分类。

加载预训练的 BERT 模型

·使用 |bert|函数加载预训练的BERT模型。该模型包含一个将文本编码为整数序列的分词器以及参数结构。

mdl =伯特

·查看BERT模型的分词器。该分词器将文本编码为整数序列，并存储填充标记、起始标记、分隔符标记及掩码标记等详细信息。

分词器 = mdl。分词器

RENMIN 中国大学

加载数据

·加载示例数据。文件 |factoryReports.csv|包含工厂报告，其中包含每个事件的文本描述及分类标签。

```
filename="factoryReports.csv";

data   =readtable(filename,"TextType","string");
```

8×5表格

头部（数据）	描述	类别：紧急处理方案		代价
	物品偶尔会卡在扫描仪滚筒中。“机械故障”” 介质		重新调整机	45
	“组装活塞发出剧烈的 rattling（rattling）声和 banging（banging）声。” 机械故障 “介质”		重新调整机	35
	启动工厂时需切断电源。	“电子故障” ”严重	“完全更换”	16200
	组装机中的油浸电容器。	“电子故障” ”严重”	“更换” 组件	352
	搅拌器导致保险丝熔断。	“电子故障 “低	添加到观察列表	55
	施工剂中的爆破管正在喷射冷却剂。	“泄漏 “高” 级	“更换” 组件	371
	混合器中的保险丝已熔断。	电子故障 “低”	“更换” 组件	441
	“物品不断从传送带上掉落下来。”	机械故障 “低”	重新调整机	38

加载数据

·本示例的目标是根据 | “类别|”列中的标签对事件进行分类。为

将数据划分为不同类别，请将这些标签转换为分类型变量。

数据。类别 = 分类型（data.Category）；

·查看课程数量。

类 = 类别（数据. 类别）；

类数量 = 数量（类）

%4

4x1细胞

	1
1	电子故障
2	泄漏；渗漏
3	机械故障
4	软件故障

加载数据

·使用直方图查看数据中各类别的分布情况。

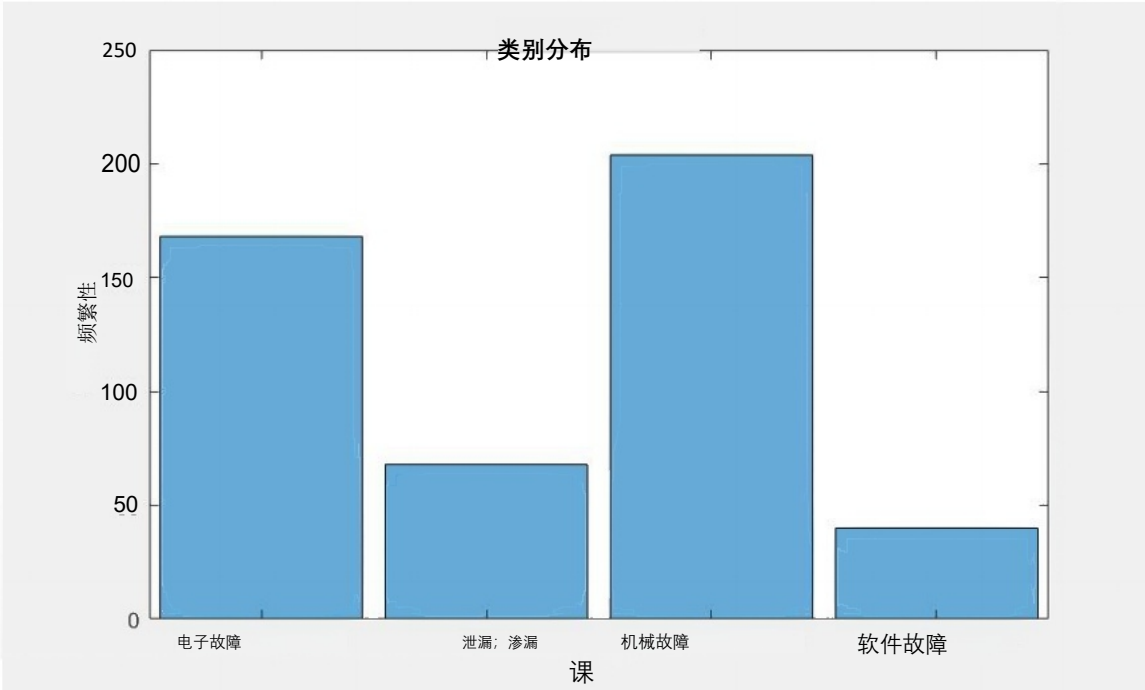
图

直方图（数据类别）；

xlabel（“类别”）

ylabel: “频率”

标题（“班级分布”）



加载数据

·使用BERT模型分词器中的 |encode|函数对文本数据进行编码，

并将生成的标记添加至训练数据表中。

数据。词元 = 编码（分词器，数据描述）；

480x6尺寸的桌子

	1 描述	2 类别	3 紧迫	4 分辨率	5 代价	6 Tokens
1	商品偶尔会出现质量问题（即出现故障或损坏）..	Mechanica.	“媒介”	重新调整M。	45	7x14尺寸的双层结构
2	剧烈的 rattling 和 banging 声	机械师.a	“媒介”	重新调整M。	35	1x14双联结构
3	电力供应出现了削减。.	电子的 ...	"高"	完全替换.....	16200	7x13 双层结构
4	组装过程中的油浸电容器	电子的 ...	"高"	更换公司。	352	1×11双倍结构
5	搅拌器导致保险丝熔断。	电子的 ...	“低”	添加到 Wa 中	55[	[102, 23229, 21130, 1997, 19977, 2016, 1013, 103]
6	施工过程中出现爆管现象.....	泄漏；渗漏	"高"	“替换 Co”.	371	7x13双倍
7	混合器中的保险丝已熔断。	电子的.	“低”	更换公司。	441	[102, 1038, 19977, 2004, 10677, 2000, 1997, 23229, 1013, 103]
8	物品仍在不断从上面掉落下来。	Mechanica ...	“低”	重新调整 M	38	7×11的双倍组合
9	输送带上掉落的物品。	Mechanica ...	“低”	重新调整M。	41[	[102, 4635, 5168, 2014, 1997, 16637, 2954, 5584, 1013, 103]
10	扫描仪卷轴已断裂，因此无法继续使用。.	Mechanica ...	“媒介”	“替换Co”.	407	1x15双倍
11	扫描仪中的保险丝已熔断。	电子的 ...	"高"	更换公司。	445	[102, 19977, 2016, 2025, 10677, 2000, 1997, 26222, 1013, 103]
12	扫描仪组件发出刺耳的尖叫声.....	电子的 ...	“低”	添加到 Wa 中	77	[102, 28350, 5391, 2014, 1997, 26222, 3275, 1013, 103]
13	排序控制器忽略了某个关键参数（int）。.	软件F...“	“低”	更新公司信息	119	[102, 4067, 2122, 11487, 19047, 2016, 2001, 8279, 1013, 103]
14	这些“sorti”装置发出的声响显得笨拙不堪。	机械师.a	“低”	添加到沃特	631x12双重	

OF CHINA

加载数据

- 将数据划分为训练集和用于验证与测试的保留集。指定保留比例为20%。

cvp=cvpartition (data. Category, “Holdout”,
0.2); dataTrain =data (training(cvp), :); %384×6
dataValidation =data (test(cvp), :); %96×6

- 查看训练集和验证集的观测值数量。numObservationsTrain = size (dataTrain, 1) %384; numObservationsValidation = size(dataValidation, 1)%96

加载数据

- 从划分后的表格中提取文本数据、标签以及编码后的BERT令牌。

textDataTrain = dataTrain. 描述;

文本数据验证 = 数据验证。描述;

TTrain= dataTrain. 类别;

验证 = 数据验证。类别;

tokensTrain = dataTrain. Tokens;

令牌验证 = 数据验证。令牌;

加载数据

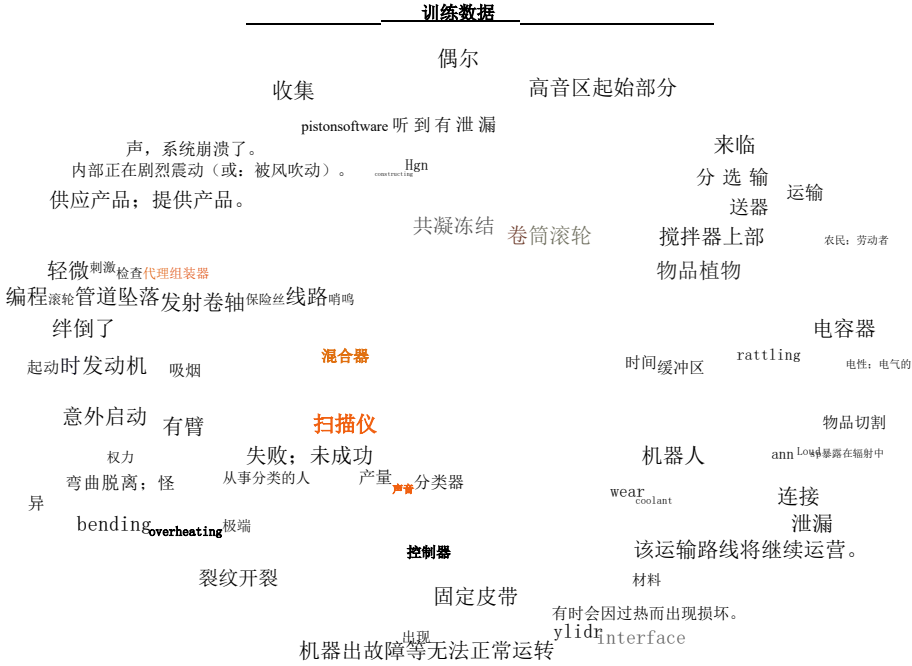
- 为确认数据已正确导入，请进行可视化查看。

使用词云对文本数据进行训练。

图

词云

题



加载数据

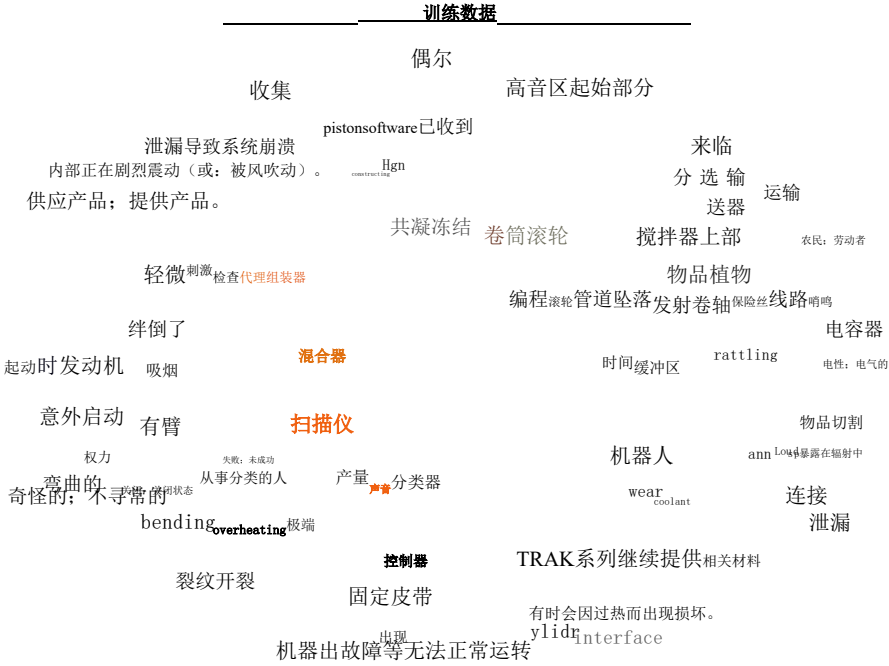
- 为确认数据已正确导入，请进行可视化查看。

使用词云对文本数据进行训练。

图

词云

题



准备用于训练的数据

- 小批量队列需要使用单一数据存储来同时输出预测结果和响应结果。创建包含BERT训练样本标记和标签的数组数据存储，并使用 |combine|函数进行合并。

```
dsXTrain =arrayDatastore(tokensTrain,"OutputType","same");
dsTTrain =arrayDatastore(TTrain);
cdsTrain =combine(dsXTrain,dsTTrain);
```

- Create a combined datastore for the validation data using the same steps.

```
dsXValidation =arrayDatastore(tokensValidation,"OutputType","same");
dsTValidation =arrayDatastore(TValidation);
cdsValidation =combine(dsXValidation,dsTValidation);
```

准备用于训练的数据

- 迷你批次队列需要使用单一数据存储来同时输出预测结果和响应结果。创建包含BERT训练样本标记和标签的数组数据存储，并使用 |combine|函数进行合并。

```
dsXTrain =arrayDatastore(tokensTrain,"OutputType","same");
dsTTrain =arrayDatastore(TTrain);
cdsTrain =combine(dsXTrain,dsTTrain);
```

- Create a combined datastore for the validation data using the same steps.

```
dsXValidation =arrayDatastore(tokensValidation,"OutputType","same");
dsTValidation =arrayDatastore(TValidation);
cdsValidation =combine(dsXValidation,dsTValidation);
```

准备用于训练的数据

- 为训练数据创建小批量队列。指定小批量大小为32，并使用|preprocessPredictors|对小批量数据进行预处理。

```
miniBatchSize =32;
paddingValue = mdl. Tokenizer. PaddingCode；%1
最大序列长度 = mdl。参数。超参数。上下文数量；%512
mbqTrain = minibatchqueue (cdsTrain，1，... %output -> 1
    “MiniBatchSize”，miniBatchSize,..
  "MiniBatchFcn",@(X)preprocessPredictors(X,paddingValue,maxSequenceLength)); mbqValid-
ation =minibatchqueue(cdsValidation,1,…
  "MiniBatchSize",miniBatchSize,...%X->1×SequenceLength×BatchSize (1×15×32)
  "MiniBatchFcn",@(X)preprocessPredictors(X,paddingValue,maxSequenceLength));
```

准备用于训练的数据

- 将BERT模型标记的训练序列转换为一个 |N|×|的特征向量嵌入维度|数组，其中 |N|表示训练样本数量， |embeddingDimension|表示BERT嵌入的维度。

```
featuresTrain =[];
reset(mbqTrain);
```

在拥有数据（mbqTrain）的情况下

```
X=下一个 (mbqTrain)；%1×15×32 数组
features =bertEmbed(X,mdl.Parameters);%768×32 dlarray
featuresTrain =[featuresTrain gather(extractdata(features))];
```

最后部分

特征=特征 Train. ‘； %384×768 dlarray

RENMIN中国大学

准备用于训练的数据

- 使用相同步骤将验证数据转换为特征向量。featuresValidation =[];

```
reset(mbqValidation);
当 hasdata (mbqValidation) X=next
    (mbqValidation)； %1×14×32 dlarray 时
    特征表示为 `bertEmbed(X，mdlParameters)`；数据结构为 `768×32` 的深度学习数组（dlarray）。

    特征验证 = cat(2，featuresValidation，gather(extractdata(features)))； end
featuresValidation  =featuresValidation.';%96×768  dlarray
```

定义深度学习网络

特征数量 = mdl. 参数。超参数。隐藏层数量： %768 层 = [

```
featureInputLayer(numFeatures)
fullyConnectedLayer(numClasses)
softmaxLayer
classificationLayer];
```

指定培训选项

- 使用 |trainingOptions|函数指定训练参数。设置小批量大小为64进行训练。每个训练轮次均随机打乱数据顺序。使用验证数据对网络进行验证。在图表中显示训练进度并隐藏详细输出信息。

选项 = trainingOptions(‘adam’ , ..)

"MiniBatchSize",64,...

验证数据：{ featuresValidation, dataValidation. category } .....

“随机打乱”、“每轮迭代一次” , ...
“情节”、“训练进度” ...

...、“详细信息” , O);

列车与测试网络

·使用 |trainNetwork|函数训练网络。

```
net=trainNetwork(featuresTrain,dataTrain.Category,layers,opts);
```

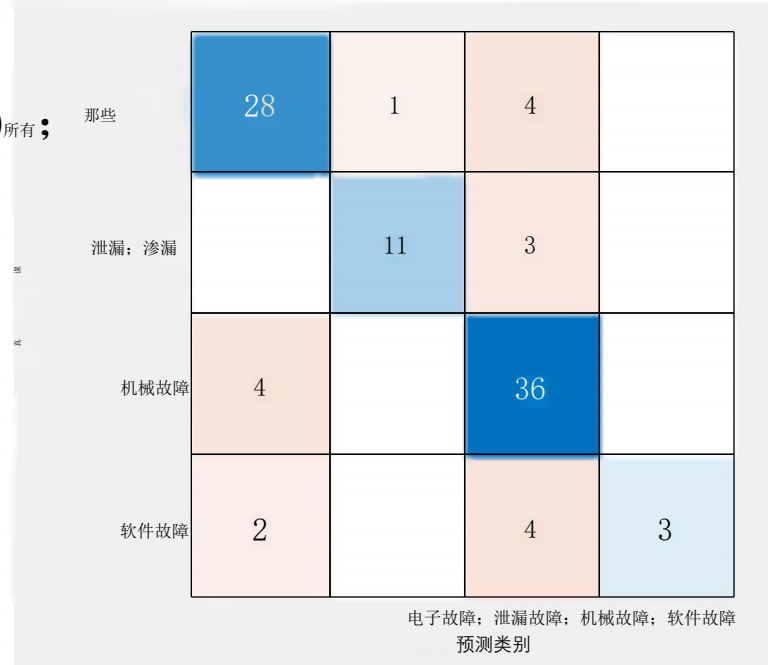
• 使用验证数据进行预测，并将结果展示出来。

混淆矩阵

YPredValidation = classify(net， featuresValida)t i o 所有；

图

```
confusionchart(TValidation,YPredValidation)
```



列车与测试网络

·计算　　那　确认　　精度

精度 = 平均值（数据验证。类别）　　==YPredValidation)%0.8125

利用新数据进行预测

·对三份新报告的事件类型进行分类，并创建一个包含这些新报告的字符串数组。

reportsNew=[…

冷却液在分选机下方积聚。

启动时，Sorter会熔断保险丝。

组装机发出一些非常响亮的咔嗒声。

·　使用与训练文档相同的步骤对文本数据进行分词处理。

tokensNew =encode(tokenizer,reportsNew);

令牌（新）

3x1 细胞

	1
1	1×11双倍组合
2	1×11双倍组合
3	1x15双倍

利用新数据进行预测

- 使用该方法将各个标记序列填充至相同长度。

lpadsequences|function and pad using the tokenizer padding code.

XNew =padsequences(tokensNew,2,"PaddingValue",tokenizer.PaddingCode);

%1×15×3 双层结构

- 使用训练好的模型对新序列进行分类。特征集：featuresNew = bertEmbed (XNew，mdlParameters)； %3×768；dlarray 特征集：featuresNew = gather (extractdata(featuresNew))； %3×768；单标签集合：singlelabelsNew = classify (net，featuresNew)。

泄漏百分比

电子故障百分比

机械故障百分比

自注意力函数

函数[A，当前]=**注意力(**X，过去数据，权重参数，超参数，nvp)%attention 全多头注意力机制

%[A，present]=attention(X，past，weights，hyperParameters)计算X上的百分比多头注意力块

%Inputs:

%　　X　　-A (numFeatures*numHeads)-by-numInputSubwords-by-numObs

%　　输入数组。

%Outputs:

%　　A　　-A(numFeatures*numHeads)-by-numInputSubwords-by-numObs

%　　输出阵列.

使用全连接层从输入数据生成查询、键值对。C=Transformer层convolution1d

(X，%768×15×32 dlarray)

重量： attn_c_attn_w_0 ， .....%2304×768（单位：分升）
重量。 attn_c attn_b_0）； %2304×1 dlarray

%2304×15×32 dlarray

将结果分为 Q（查询）、K（键）和 V（值）。

分割大小 = 大小（C，1）/3；%768

Q=C(1:splitSize,:,:); %768×15×32 dlarray

K=C((splitSize+1):(2*splitSize),:,:); %768×15×32 dlarray

V=C((2*splitSize+1):(3*splitSize),:,:);%768×15×32 dlarray

%分割标题

Q=分割头数组（Q，分割大小，超参数。 NumHeads)；%64×15×12×32： dlarray）

K=分裂头数（K，分裂大小，超参数。NumHeads）； %64×15×12×32 dl数组

V=分割头部函数（参数：V、分割大小、超参数。 NumHeads)；%64×15×12×32： dlarray）

A=transformer.layer.multiheadAttention(Q,K,V,'CausalMask',nvp.CausalMask,
'Dropout',nvp.Dropout,
'InputMask',nvp.InputMask);

%64×15×12×32 dlarray

A=iMergeHeads(A);

%768×15×32 dlarray

A = 变形器。layer(convolution1d.(A, %768×15×32 dlarray)
重量: attn_c_proj_w_0 ..... %768×768 dlarray
weights.attn_c_proj_b_0); %768×1 dlarray

最后部分

%768×15×32 dlarray

多头自注意力函数

函数 A=**多头注意力机制 (**Q，K，V，nvp)

%多头注意力机制 Multi-head Attention %

% A= 多头注意力机制（Q，K，V）用于计算针对多个注意力头的归一化点积注意

力。该函数为提升效率而一次性计算多个注意力头的注意力值。Q 是查询集合。%matrices，K 是一组关键矩阵，V 是一组%value 矩阵。输出结果 A 是

一组注意力矩阵。

我们通过计算Q矩阵与K矩阵的乘积来确定注意力权重。W是 numAllSubWords-by-numInputSubWords-by-numHeads. 矩阵，其每个元素均为Q中的查询向量与K中的键向量之间的点积。

W = dlmtimes (移位(K，[2134])，Q)；%64×15×12×32　　dlarray

%15×15×12×32　　dlarray

%除以 d 的平方根

W = √(size(Q，1))；

应用遮罩

W=transformer.layer.maskAttentionWeights(W,'CausalMask',nvp.CausalMask,'InputMask',nvp.InputMask);

%15×15×12×32　　dlarray

应用softmax函数

```
W=softmax(W,'DataFormat','CTUB');
%15×15×12×32        dlarray
```

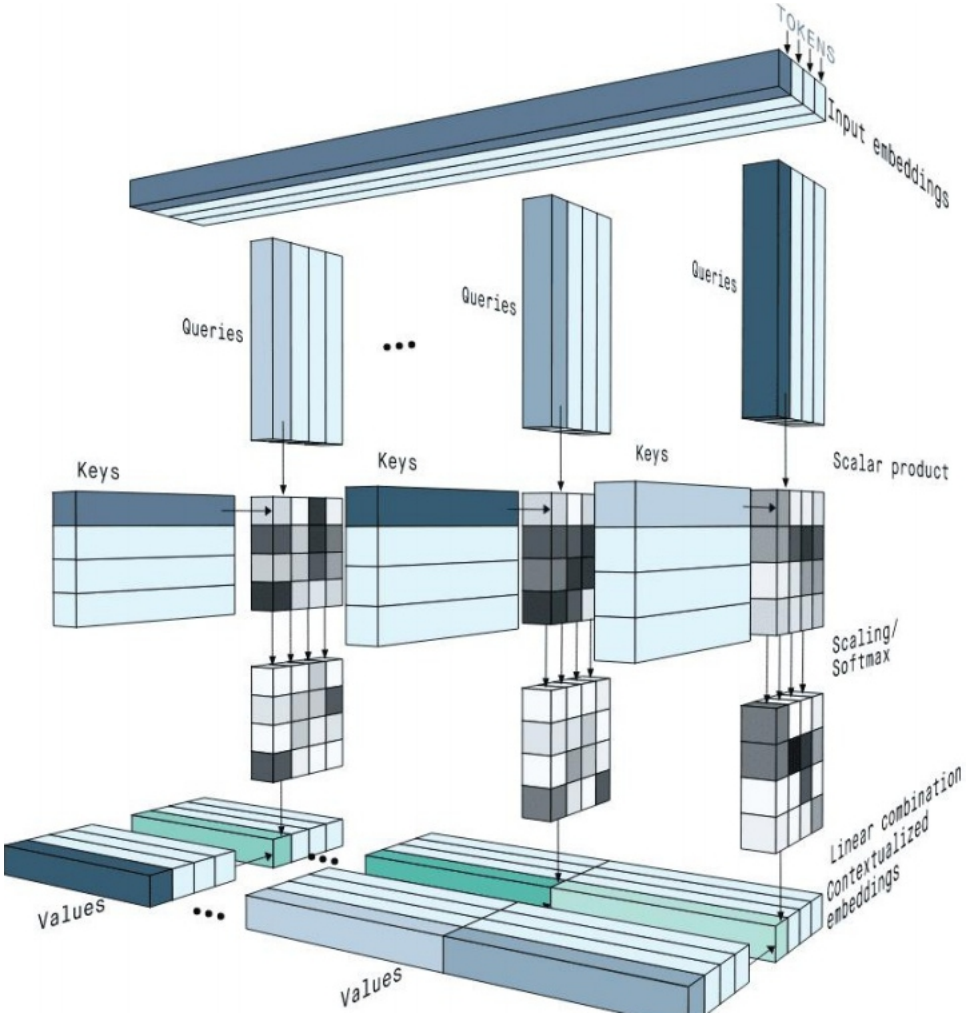
%应用丢弃

```
W = transformer  layer_dropout(W，  nvp. Dropout)；
```

我们通过计算注意力权重 %W 与 V 的乘积来得出注意力值。A 的维度为特征数量×输入子词数量×头数量。对 A的一种解释是：它表示根据由 W 给出的概率分布，V 的期望值。

```
A=dlmtimes (V，  W)；  %V:64×15×12×32 dlarray W:15×15×12×32 dlarray %64×15×12×32 dlarray
```

最后部分



Values

RENMIN 中国大学